

Python API Overview

2026-05-04

目录

Python API Overview	9
Import Pattern	9
Coverage Status	9
PDF Export	9
phynexis.fields	10
Module Status	10
Submodules	10
See also	11
CSRMatrix	11
Description	11
Constructors	11
Methods	12
Unexposed C++ API	15
Known Issues	16
FieldBase	16
Description	16
Constructors	16
Methods	16
Properties	19
Unexposed C++ API	19
Known Issues	19
FieldHolder	20
Constructors	20
Methods	21
Examples	21
Known Issues	23
Unexposed C++ API	23
FieldLayout	23
Description	23
Constructors	23
Methods	24
Known Issues	25
FieldManager	25
Description	25
Constructors	25

Methods	26
Unexposed C++ API	32
Known Issues	32
FieldMeta	33
Enums	33
Constructors	33
Properties	34
Example	34
Unexposed C++ API	34
FieldSchema	35
Description	35
Constructors	35
Methods	35
Unexposed C++ API	38
Known Issues	38
FieldSlot	38
Description	38
Constructors	38
Factory Methods	39
Properties	41
Methods	41
Field Views	42
Type Summary	42
Common API (FieldViewBase)	42
ScalarFieldView	43
Vec3FieldView	44
Vec4FieldView	45
Vec6FieldView	45
VecNFieldView	46
VecXFieldView	46
Known Issues	47
Field / ScalarField	47
Constructors	48
NumPy Interop	48
Element Access	49
Capacity	50
Bulk Operations	51
Arithmetic (Double only)	52
Comparison Semantics	52

Properties	53
Type Aliases and Integer / Bool Fields	53
Known Issues	53
Unexposed C++ API	54
I/O and Utilities	54
Console Output	54
Field I/O	55
Manager I/O	56
VecXField I/O	56
MPI Utilities	57
Known Issues	58
LinkedFieldView	58
Related Types	58
LinkedFieldView	59
LinkedFieldRowView	60
Attached Data Adapters	60
Known Issues	61
Operators	61
Math	61
Reduction	63
Prefix Sum	63
Sort	64
Radix Sort	65
Known Issues	65
phynexis.parsim	65
Module Status	66
Submodules	66
Quick Start	66
See also	66
Graph	67
ComputationalGraph	67
SetBase	67
NodeSet	68
EdgeSet	69
HyperEdgeSet	69
ShapeStore	69
Views	71
Known Issues	71
Models and Utils	72

RepulsionModel	72
LinearRepulsionModel	72
RepulsionModelFactory	72
NodeGenerator	73
FieldSampler	74
Node Property Utils	74
SpatialIndex / UniformGridIndex	74
Known Issues	75
Schemas	75
Schema Types	75
BoundLayout	76
Known Issues	77
Simulator	77
Simulator	77
SimulationConfig	78
Context	78
OperatorSystem	79
RuntimeState	79
DomainSystem	79
SpatialIndex	80
UniformGridIndex	80
ResourceRegistry / ContextResource	81
Known Issues	81
phynexis.utils	81
Import	81
Module Overview	82
C++ Namespace	82
pybind Module	82
See also	82
Console	83
Config	83
Logging Functions	84
Control Functions	85
Cork	85
Description	85
Methods	86
Note	86
Unexposed C++ API	86
DataIO	86

Methods	86
Example	87
Unexposed C++ API	87
FlatHashMap	88
Available Instantiations	88
Constructor	88
Methods	88
Python Protocols	89
Example	89
Unexposed C++ API	91
Binding Fix Record	91
LevelSetField	92
Constructor	92
Methods	92
Example	93
Unexposed C++ API	94
math	94
Constants	94
Vector Operations	94
Quaternion	95
Example	95
Unexposed C++ API	95
MiniMap	96
Constructor	96
Methods	96
Python Protocols	96
Example	96
Unexposed C++ API	97
OpenMP	97
Functions	97
Example	98
Note	98
sampling	98
Distribution Sampling	98
Spatial Sampling	98
Orientation Sampling	99
Spherical Sampling	99
Example	99
Unexposed C++ API	99

SaveOptions / LoadOptions	100
FileFormat	100
SaveOptions	100
SaveVtkOptions	100
LoadOptions	101
Example	101
SDFCalculator	102
Constructor	102
Methods	102
Example	103
Unexposed C++ API	103
STLModel	103
Constructor	104
Properties	104
Methods	104
Static Methods	105
Example	105
Example: Create from vertices and facets	106
Unexposed C++ API	106
StringUtils	106
Functions	106
Unexposed C++ API	107
SurfaceMesh	107
Constructor	107
Properties	107
Example	108
Unexposed C++ API	108
TimeUtils	108
Functions	108
Unexposed C++ API	109
Vectors and Matrices	109
Vector Types	109
Matrix Types	110
Example	110
Known Issues	111
Unexposed C++ API	112
Voronoi	112
TessellationParams	112
Tessellation	112

Diagram	113
Example	113
Unexposed C++ API	113
vtk	113
Data Classes	114
Read Functions	114
Write Functions	114
Format Detection	114
Example	114
Known Issues	115
Unexposed C++ API	115
wrappers	115
CorkWrapper	115
EigenWrapper	115
IGLWrapper	115
CGALWrapper	116
mpi	116
Example	116
Known Issues	117
Unexposed C++ API	117

Python API Overview

Use the header **Docs** item for the [manual / guides](#). On API pages the left sidebar shows **only** this Python reference tree (no manual outline).

Phynexis exposes its C++ core through pybind11 bindings. The table below lists the **nine lazy-loaded Python submodules** in a typical build. **On this documentation site**, detailed reference pages are maintained primarily under `utils` and `fields` (plus historical notes under `deprecated`). Other rows describe modules that ship with the package; page-level coverage may still be thin—use the [coverage tracker](#) and the [PDF export](#) for the broadest automated snapshot.

Module	Python Import	C++ Namespace	Description
utils	<code>phynexis.utils</code>	<code>phynexis::utils</code>	Utilities: console , SDF , string helpers
fields	<code>phynexis.fields</code>	<code>phynexis::fields</code>	Field quantities and level sets
parsim	<code>phynexis.parsim</code>	<code>phynexis::parsim</code>	Parallel simulation framework
netdem	<code>phynexis.netdem</code>	<code>phynexis::netdem</code>	Discrete Element Method (DEM)
netfem	<code>phynexis.netfem</code>	<code>phynexis::netfem</code>	Finite Element Method (FEM)
cfddem	<code>phynexis.cfddem</code>	<code>phynexis::cfddem</code>	CFD-DEM coupling
ml	<code>phynexis.ml</code>	<code>phynexis::ml</code>	Machine learning interfaces
peridigm	<code>phynexis.peridigm</code>	<code>phynexis::peridigm</code>	Peridynamics
workflow	<code>phynexis.workflow</code>	<code>phynexis::workflow</code>	Workflow automation

Import Pattern

```

1 import phynexis
2
3 # Lazy-loaded submodules
4 scene = phynexis.netdem.Scene()
5 mesh = phynexis.netfem.TetMesh()
```

python

Coverage Status

See [progress tracker](#) for the current documentation coverage by module.

PDF Export

A compiled PDF merges the same Markdown tree (including pages with sparse prose). Regenerate locally after large binding updates:

```
python scripts/export-python-api-pdf.py
```

Published build:

[Download phynexis-python-api.pdf](#)

phynexis.fields

Python: `phynexis.fields` **pybind module:** `pyfields`

Field data structures for structured and unstructured grids. Provides typed scalar/vector fields, field layouts, views, schemas, and mathematical operators.

Module Status

Class	Status	Doc
<code>FieldMeta</code>	done	field-meta
<code>FieldType</code>	done	field-meta
<code>ValueType</code>	done	field-meta
<code>Field</code> / <code>ScalarField</code> / <code>Int32Field</code> / <code>Int64Field</code> / <code>BoolField</code>	done	field
<code>FieldHolder</code>	done	field-holder
<code>FieldBase</code>	done	field-base
<code>FieldLayout</code>	done	field-layout
<code>FieldManager</code>	done	field-manager
<code>FieldSlot</code>	done	field-slot
<code>FieldSchema</code>	done	field-schema
<code>CSRMatrix</code>	done	csr-matrix
<code>FieldViewBase</code> / <code>ScalarFieldView</code> / <code>Vec3FieldView</code> / <code>Vec4FieldView</code> / <code>Vec6FieldView</code> / <code>VecNFieldView</code> / <code>VecXFieldView</code>	done	field-views
<code>LinkedFieldView</code> / <code>LinkedFieldRowView</code>	done	linked-field-view
Operators (<code>math</code> , <code>reduction</code> , <code>prefix_sum</code> , <code>sort</code> , <code>radix_sort</code>)	done	operators
I/O and Utilities (VTK, binary, JSON, MPI, console)	done	io-utils
Others	pending	—

Submodules

- **core** — `Field`, `FieldMeta`, `FieldLayout`, `FieldManager`, `FieldSchema`, `FieldSlot`, `FieldBase`, `FieldHolder`
- **containers** — `VecXField`, `CSRMatrix`

- **views** — `ScalarFieldView`, `Vec3FieldView`, `VecXFieldView`, `LinkedFieldView`, `FieldViewBase`
- **schema** — `ScalarFieldSchema`, `Vec3FieldSchema`, `Vec4FieldSchema`, `Vec6FieldSchema`, `VecNFieldSchema`, `VecXFieldSchema`
- **operators** — Math, reduction, prefix sum, sort, radix sort
- **utils** — I/O, MPI, console output

See also

- [Manual home](#) — installation, simulations, visualization
- [Python API overview](#) — all submodules at a glance
- [Deprecated flat references](#) — legacy v0 pages

CSRMatrix

C++: `phynexis::fields::CSRMatrix<ValueType, IndexType>` **Python:** `phynexis.fields.CSRMatrix` (alias `CSRMatrixd`), `phynexis.fields.CSRMatrixi` **Header:** `src/fields/containers/csr_matrix.hpp`

Description

`CSRMatrix` is a compressed sparse row (CSR) matrix for linear algebra operations. Each row stores `(column, value)` pairs sorted by column index. The class supports dynamic element insertion, row-wise queries, matrix-matrix and matrix-vector multiplication, and transpose.

Two concrete types are exposed:

Python Type	Value Type	Index Type
<code>CSRMatrixd</code> / <code>CSRMatrix</code>	<code>double</code>	<code>Int64</code>
<code>CSRMatrixi</code>	<code>Int64</code>	<code>Int64</code>

Constructors

`CSRMatrix()`

Creates an empty 0×0 matrix.

`CSRMatrix(num_rows, num_cols)`

Creates a matrix with the given dimensions. All entries are initially zero (not stored).

Parameters: | Parameter | Type | Description | |----|----|-----| | `num_rows` | `int` | Number of rows | | `num_cols` | `int` | Number of columns |

Example:

```
1 import phynexis
2 m = phynexis.fields.CSRMatrix(3, 3)
3 print(m.get_num_rows(), m.get_num_cols())
```

python

Output:

1 3 3

text

Methods

`resize(num_rows, num_cols)`

Resizes the matrix. Existing data is preserved where possible.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `num_rows` | `int` | New row count | | `num_cols` | `int` | New column count |

`set_element(row, col, value)`

Sets a single element. If the element already exists, its value is overwritten.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `row` | `int` | Row index | | `col` | `int` | Column index | | `value` | `float` / `int` | Value to set |

Example:

```
1 import phynexis
2 m = phynexis.fields.CSRMatrix(3, 3)
3 m.set_element(0, 0, 1.0)
4 m.set_element(1, 1, 2.0)
5 m.set_element(2, 2, 3.0)
6 print(m.get_element(0, 0), m.get_element(1, 1))
```

python

Output:

1 1.0 2.0

text

`get_element(row, col)`

Returns the value at `(row, col)`. Returns 0.0 if the element is not explicitly stored.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `row` | `int` | Row index | | `col` | `int` | Column index |

Returns: `float` / `int`

`has_element(row, col)`

Returns `True` if the element is explicitly stored (non-zero and inserted).

Parameters: | Parameter | Type | Description | |-----|----|-----| | `row` | `int` | Row index | | `col` | `int` | Column index |

Returns: `bool`

`set_row(row, cols, vals)`

Replaces the entire row with the given column indices and values.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `row` | `int` | Row index | | `cols` | `list[int]` | Column indices | | `vals` | `list[float]` | Corresponding values |

Example:

```
1 import phynexis
```

python

```

2 m = phynexis.fields.CSRMatrix(3, 3)
3 m.set_row(0, [0, 2], [1.0, 3.0])
4 print(m.get_row(0))
5 print(m.get_row_indices(0))

```

Output:

```

1 [1.0, 3.0]
2 [0, 2]

```

text

get_row(row)

Returns the non-zero values in the given row.

Parameters: | Parameter | Type | Description | |-----|----|-----| | row | int | Row index |

Returns: list[float] / list[int]

get_row_indices(row)

Returns the column indices of non-zero values in the given row.

Parameters: | Parameter | Type | Description | |-----|----|-----| | row | int | Row index |

Returns: list[int]

get_row_size(row)

Returns the number of non-zero elements in the given row.

Parameters: | Parameter | Type | Description | |-----|----|-----| | row | int | Row index |

Returns: int

set_elements(rows, cols, vals)

Batch-sets multiple elements.

Parameters: | Parameter | Type | Description | |-----|----|-----| | rows | list[int] | Row indices | | cols | list[int] | Column indices | | vals | list[float] | Values |

Example:

```

1 import phynexis
2 m = phynexis.fields.CSRMatrix(2, 2)
3 m.set_elements([0, 1], [1, 0], [5.0, 7.0])
4 print(m.get_row(0), m.get_row(1))

```

python

Output:

```

1 [5.0] [7.0]

```

text

add_elements(rows, cols, vals)

Adds values to existing elements. If an element does not exist, it is created.

Parameters: | Parameter | Type | Description | |-----|----|-----| | rows | list[int] | Row indices | | cols | list[int] | Column indices | | vals | list[float] | Values to add |

Example:

```

1 import phynexis

```

python

```

2 m = phynexis.fields.CSRMatrix(2, 2)
3 m.set_elements([0], [0], [10.0])
4 m.add_elements([0], [0], [5.0])
5 print(m.get_element(0, 0))

```

Output:

```
1 15.0
```

text

multiply(other)

Matrix-matrix multiplication. Returns a new `CSRMatrix`.

Parameters: | Parameter | Type | Description | |-----|----|-----| | other | `CSRMatrix` | Right-hand side matrix |

Returns: `CSRMatrix`

multiply(vector)

Matrix-vector multiplication.

Parameters: | Parameter | Type | Description | |-----|----|-----| | vector | `list[float]` | Dense vector (length must match column count) |

Returns: `list[float]`

Example:

```

1 import phynexis
2 m = phynexis.fields.CSRMatrix(2, 2)
3 m.set_elements([0, 1], [0, 1], [2.0, 3.0])
4 v = m.multiply([1.0, 1.0])
5 print(v)

```

python

Output:

```
1 [2.0, 3.0]
```

text

transpose()

Returns the transpose of the matrix.

Returns: `CSRMatrix`

Example:

```

1 import phynexis
2 m = phynexis.fields.CSRMatrix(2, 3)
3 m.set_element(0, 1, 5.0)
4 mt = m.transpose()
5 print(mt.get_num_rows(), mt.get_num_cols())
6 print(mt.get_element(1, 0))

```

python

Output:

```

1 3 2
2 5.0

```

text

`get_num_rows()` / `get_num_cols()`

Returns the matrix dimensions.

Returns: `int`

`get_total_elements()`

Returns the total number of stored elements (including duplicates if any).

Returns: `int`

`get_nnz()`

Returns the number of non-zero elements.

Returns: `int`

`get_sparsity()`

Returns the sparsity ratio $(1 - \text{nnz} / (\text{rows} * \text{cols}))$.

Returns: `float`

`get_memory_usage()`

Returns the estimated memory usage in bytes.

Returns: `int`

`optimize_memory()`

Reclaims unused capacity in all rows.

`size()`

Returns the total number of stored elements. Same as `get_total_elements()`.

Returns: `int`

`empty()`

Returns `True` if the matrix contains no stored elements.

Returns: `bool`

Unexposed C++ API

The following C++ APIs are not exposed via pybind:

- `clear()` — removes all elements
- `row(i)` — returns a row view (`CSRMatrixRowView`)
- `get_row_data_ptr(i)` — returns raw row data pointer

Known Issues

`clear()` not exposed

The `clear()` method is not bound to Python. To clear a matrix, create a new instance or use `resize(0, 0)` as a workaround.

Status: noted

FieldBase

C++: `phynexis::fields::FieldBase` **Python:** `phynexis.fields.FieldBase` **Header:** `src/fields/core/field_base.hpp`

Description

`FieldBase` is the abstract base class for all field containers in `phynexis`. It defines the common interface shared by `ScalarField`, `Int32Field`, `Int64Field`, `BoolField`, and all other typed field specializations.

Important: `FieldBase` cannot be instantiated directly in Python (or C++). Use concrete subclasses such as `ScalarField` instead.

Known Issue: Due to a `pybind11` binding defect, `Field<T>` is not currently registered as a subclass of `FieldBase`. As a result, `FieldBase`-only methods (`ensure_capacity`, `erase_indices`, `reorder`, `swap_elements`, `copy_element`, `reset_value`, `is_same_type`) are **unreachable from any concrete field instance**. The methods that do work (`size`, `empty`, `resize`, `reserve`, `clear`, `name`) are available because each subclass re-binds them independently.

Constructors

`FieldBase` has no exposed constructor. It is an abstract class.

Methods

`size()`

Returns the number of elements in the field.

Returns: `int` — element count

Example:

```
1 import phynexis
2 sf = phynexis.fields.ScalarField("pressure", 100, 0.0)
3 print(sf.size())
```

python

Output:

```
1 100
```

text

`empty()`

Returns `True` if the field contains no elements.

Returns: bool**Example:**

```
1 import phynexis
2 sf = phynexis.fields.ScalarField("empty_field", 0, 0.0)
3 print(sf.empty())
```

python

Output:

```
1 True
```

text

resize(new_size, value=None)

Changes the number of elements. If `value` is provided, new elements are initialized to it; otherwise default construction is used.

Parameters: | Parameter | Type | Default | Description | |-----|---|-----|-----| | `new_size` | int | — | New element count | | `value` | float / int / bool | None | Value for new elements |

Example:

```
1 import phynexis
2 sf = phynexis.fields.ScalarField("x", 3, 1.0)
3 sf.resize(5)
4 print("size after resize:", sf.size())
5 sf.resize(5, 7.0)
6 print("values:", [sf[i] for i in range(5)])
```

python

Output:

```
1 size after resize: 5
2 values: [1.0, 1.0, 1.0, 7.0, 7.0]
```

text

reserve(capacity)

Pre-allocates internal storage for at least `capacity` elements without changing the logical size.

Parameters: | Parameter | Type | Description | |-----|---|-----| | `capacity` | int | Minimum storage capacity |

clear()

Removes all elements, setting size to 0.

Example:

```
1 import phynexis
2 sf = phynexis.fields.ScalarField("x", 5, 1.0)
3 sf.clear()
4 print(sf.size(), sf.empty())
```

python

Output:

```
1 0 True
```

text

ensure_capacity(total_needed)

Ensures the field can hold at least `total_needed` elements. Equivalent to `reserve(total_needed)` if capacity is insufficient.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `total_needed` | `int` | Required minimum capacity |

Status: ⚠️ Unreachable — see Known Issues below.

`erase_indices(idx_sorted_unique)`

Removes elements at the given indices. Indices must be sorted in ascending order and unique.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `idx_sorted_unique` | `list[int]` | Sorted unique indices to remove |

Status: ⚠️ Unreachable — see Known Issues below.

`reorder(new_to_old)`

Reorders elements in-place according to the permutation map.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `new_to_old` | `list[int]` | For each new position `i`, `new_to_old[i]` gives the old index |

Status: ⚠️ Unreachable — see Known Issues below.

`swap_elements(i, j)`

Swaps the elements at indices `i` and `j`.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `i` | `int` | First index | | `j` | `int` | Second index |

Status: ⚠️ Unreachable — see Known Issues below.

`copy_element(src_index, dst_index)`

Copies the value at `src_index` to `dst_index`.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `src_index` | `int` | Source index | | `dst_index` | `int` | Destination index |

Status: ⚠️ Unreachable — see Known Issues below.

`reset_value(index)`

Resets the element at `index` to its type's default value (`0.0`, `0`, or `False`).

Parameters: | Parameter | Type | Description | |-----|----|-----| | `index` | `int` | Index to reset |

Status: ⚠️ Unreachable — see Known Issues below.

`is_same_type(other)`

Returns `True` if `other` has the same `ValueType` and `FieldType`.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `other` | `FieldBase` | Another field to compare with |

Returns: `bool`

Status: ⚠️ Unreachable — see Known Issues below.

Properties

Property	Type	Access	Description
name	str	read/write	Field name identifier

Example:

```
1 import phynexis
2 sf = phynexis.fields.ScalarField("old_name", 10, 0.0)
3 print(sf.name)
4 sf.name = "new_name"
5 print(sf.name)
```

python

Output:

```
1 old_name
2 new_name
```

text

Unexposed C++ API

The following C++ APIs are not exposed via pybind:

- `append(other)` — append another field of the same type
- `reset_values()` — reset all elements to default
- `set_value(index, void* value)` — untyped value setter (not Python-friendly)
- `get_value_ptr(index)` — returns untyped pointer (not Python-friendly)
- `value_type()` — returns `ValueType` enum
- `field_type()` — returns `FieldType` enum
- `subset(mask)` — extract elements where `mask[i]` is `True`; returns `FieldHolder`
- `subset_inplace(mask)` — in-place subset
- `to_binary(out) / from_binary(in)` — binary serialization
- `pack(out, indices) / unpack(in, target_indices)` — selective binary I/O

Known Issues

`Field<T>` not registered as subclass of `FieldBase`

`pyfield.cpp` uses `pybind11::class_<Field<T>>(m, name)` without declaring `FieldBase` as the base class. As a result:

- `isinstance(scalar_field, FieldBase)` returns `False`
- `FieldBase`-only methods bound in `pyfield_base.cpp` are invisible on concrete field instances
- The methods that **do** work on subclasses (`size`, `empty`, `resize`, `reserve`, `clear`, `name`) are available because `pyfield.cpp` re-binds them independently

Workaround: None. Requires binding fix in `phynexis/bindings/fields/core/pyfield.cpp`:

```
1 // Current (broken)
2 pybind11::class_<Field<T>>(m, name.c_str(), pybind11::buffer_protocol())
3
4 // Fixed
5 pybind11::class_<Field<T>, FieldBase>(m, name.c_str(), pybind11::buffer_protocol())
```

cpp

Once fixed, the duplicated bindings of `size`, `empty`, `resize`, `reserve`, `clear`, `name` on `Field<T>` can be removed (inherited from `FieldBase`).

File: phynexis/bindings/fields/core/pyfield.cpp **Status:** open — tracked in binding-repair-backlog.md

FieldHolder

C++: phynexis::fields::FieldHolder **Python:** phynexis.fields.FieldHolder **Header:** src/fields/core/field_holder.hpp

Owning move-only smart pointer around a heap-allocated `FieldBase`. C++ code uses `FieldHolder` to return polymorphic field instances by value while keeping ownership clear (it deletes the inner pointer on destruction). The most common way to obtain one from Python is via `Field<T>.subset(mask)`.

`FieldHolder` is non-copyable; the move constructor is not exposed to Python, so treat each holder as single-use.

Constructors

Signature	Description
<code>FieldHolder()</code>	Empty holder; <code>get()</code> is <code>None</code> and <code>bool(h)</code> is <code>False</code> .
<code>FieldHolder(ptr)</code>	Adopts an existing <code>FieldBase*</code> . Pass <code>None</code> to construct an empty holder.

Practical sources of holders:

- `field.subset(mask)` — returns a new holder containing the masked subset.

Example:

```

1 import numpy as np
2 import phynexis
3
4 F = phynexis.fields
5
6 empty = F.FieldHolder()
7 print("empty bool:", bool(empty), "get():", empty.get())
8
9 sf = F.ScalarField.from_numpy(np.array([10.0, 20.0, 30.0, 40.0, 50.0]))
10 mask = F.BoolField.from_numpy(np.array([True, False, True, False, True]))
11 holder = sf.subset(mask)
12 print("holder bool:", bool(holder))
13 print("inner field:", holder.get().to_numpy())

```

python

Output:

```

1 empty bool: False get(): None
2 holder bool: True
3 inner field: [10. 30. 50.]

```

text

Methods

`get()`

Returns the contained field without transferring ownership. The Python object returned reflects the **runtime** type of the field (e.g. `ScalarField`), so you can call its full interface directly. Returns `None` if the holder is empty.

Returns: `ScalarField` / `Int32Field` / `Int64Field` / `BoolField` / `None`.

`release()`

Transfers ownership of the inner field out of the holder. After release the holder is empty (`bool(holder)` is `False`) and the returned object is owned by the Python interpreter — it remains alive even if the holder is garbage collected.

Returns: the inner field (typed concretely, e.g. `ScalarField`) or `None` if the holder was already empty.

`reset(ptr=None)`

Replaces the inner pointer. The previous content (if any) is deleted. Calling `reset()` with no argument or `reset(None)` empties the holder.

Parameters:

Parameter	Type	Default	Description
<code>ptr</code>	<code>FieldBase</code> or <code>None</code>	<code>None</code>	New pointer to adopt.

`as_field_double()`

Down-cast the inner field to `Field<double>` (alias `ScalarField`). Raises `RuntimeError("std::bad_cast")` if the inner field is not a double field, or if the holder is empty.

Returns: `ScalarField`.

`as_field_int()`

Down-cast the inner field to `Field<Int32>` (alias `Int32Field`). Raises `RuntimeError("std::bad_cast")` on type mismatch or empty holder.

Returns: `Int32Field`.

`__bool__()`

`True` when the holder owns a non-null pointer.

Examples

Lifetime semantics

`get()` returns a borrowed reference — the holder still owns the data, so the returned field becomes invalid after the holder is collected. `release()` transfers ownership to the returned object and is safe to keep beyond the holder's lifetime.

```

1 import numpy as np
2 import phynexis
3
4 F = phynexis.fields
5
6 sf = F.ScalarField.from_numpy(np.array([10.0, 20.0, 30.0]))
7 mask = F.BoolField.from_numpy(np.array([True, False, True]))

```

python

```

8 h = sf.subset(mask)
9
10 inner = h.release()
11 print("released:", inner.to_numpy())
12
13 del h # safe - ownership already transferred
14 print("still alive after del h:", inner.to_numpy())
15
16 # Re-releasing an empty holder yields None
17 print("second release:", h.release()) # noqa: F821

```

(Re-using `h` here would actually fail because `del h` removes the binding — the comment is illustrative.)

Down-cast and type mismatch

```

1 import numpy as np
2 import phynexis
3
4 F = phynexis.fields
5
6 double_holder = F.ScalarField.from_numpy(
7     np.array([1.0, 2.0, 3.0])
8 ).subset(F.BoolField.from_numpy(np.array([True, True, True])))
9
10 int_holder = F.Int32Field.from_numpy(
11     np.array([10, 20, 30], dtype=np.int32)
12 ).subset(F.BoolField.from_numpy(np.array([True, False, True])))
13
14 print("as double:", double_holder.as_field_double().to_numpy())
15 print("as int:", int_holder.as_field_int().to_numpy())
16
17 # Wrong cast raises bad_cast
18 try:
19     double_holder.as_field_int()
20 except RuntimeError as e:
21     print("wrong cast:", repr(e))

```

python

Output:

```

1 as double: [1. 2. 3.]
2 as int: [10 30]
3 wrong cast: RuntimeError('std::bad_cast')

```

text

Reset

```

1 import numpy as np
2 import phynexis
3
4 F = phynexis.fields
5
6 sf = F.ScalarField.from_numpy(np.array([1.0, 2.0]))
7 mask = F.BoolField.from_numpy(np.array([True, True]))
8 h = sf.subset(mask)
9 print("before reset:", bool(h))
10
11 h.reset() # equivalent to reset(None)
12 print("after reset():", bool(h))

```

python

Output:

```
1 before reset: True
2 after reset(): False
```

text

Known Issues

- The casts only cover `double` and `Int32`. There is no `as_field_int64()` or `as_field_bool()`, so `Int64Field` / `BoolField` subsets are reachable only through `get()`.
- `subset()` does not propagate the source field's name; the resulting field has `name == ""`.
- Because `Field<T>` is not registered as a subclass of `FieldBase` (see [binding-repair-backlog.md](#)), passing a generic `FieldBase`-typed reference back to Python loses no information in practice — `pybind` already resolves to the concrete subclass via runtime type info — but `isinstance(holder.get(), FieldBase)` is still `False`.

Unexposed C++ API

- `template <typename T> T &as()` — generic templated cast; only the `Double` and `Int32` instantiations are exposed via the named `as_field_double` / `as_field_int` lambdas.
- Move constructor / move assignment — not bound (`pybind` does not need them for ownership transfer; `release()` is the Python-visible equivalent).

FieldLayout

C++: `phynexis::fields::FieldLayout` (alias for `VecX<FieldMeta>`) **Python:** `phynexis.fields.FieldLayout`

Header: `src/fields/core/field_layout.hpp`

Description

`FieldLayout` is a sequential container of `FieldMeta` objects. It describes the structure of a group of fields — for example, which fields exist in a simulation domain, their types, and their value types. Each element is a `FieldMeta` instance.

In C++, `FieldLayout` is a type alias for `VecX<FieldMeta>`. In Python it is exposed as a standalone class with list-like semantics.

Constructors

`FieldLayout()`

Creates an empty layout.

Example:

```
1 import phynexis
2 fl = phynexis.fields.FieldLayout()
3 print(len(fl), fl.empty())
```

python

Output:

```
1 0 True
```

text

FieldLayout(metas)

Creates a layout from a list of `FieldMeta` objects.

Parameters: | Parameter | Type | Description | |----|---|----| | metas | list[FieldMeta] | Initial field metadata entries |

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 metal = F.FieldMeta("pressure", F.FieldType.Scalar, F.ValueType.Double)
5 meta2 = F.FieldMeta("velocity", F.FieldType.VecX, F.ValueType.Double)
6 fl = F.FieldLayout([metal, meta2])
7 print(len(fl))
```

python

Output:

```
1 2
```

text

Methods

__len__()

Returns the number of `FieldMeta` entries.

Returns: int

__getitem__(index)

Returns the `FieldMeta` at index.

Parameters: | Parameter | Type | Description | |----|---|----| | index | int | Zero-based index |

Returns: `FieldMeta`

Note: Negative indices are **not supported** and raise `IndexError`.

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 meta = F.FieldMeta("temperature", F.FieldType.Scalar, F.ValueType.Double)
5 fl = F.FieldLayout([meta])
6 print(fl[0])
```

python

Output:

```
1 <FieldMeta object at 0x...>
```

text

size()

Same as `len(layout)`. Returns the number of entries.

Returns: int

empty()Returns `True` if the layout contains no entries.**Returns:** `bool`**append(meta)**Appends a `FieldMeta` to the end of the layout.**Parameters:** | Parameter | Type | Description | |----|---|-----| | meta | `FieldMeta` | Metadata to append |**Example:**

```

1 import phynexis
2 F = phynexis.fields
3
4 fl = F.FieldLayout()
5 fl.append(F.FieldMeta("x", F.FieldType.Scalar, F.ValueType.Double))
6 fl.append(F.FieldMeta("y", F.FieldType.Scalar, F.ValueType.Double))
7 print(fl.size())

```

python

Output:

```

1 2

```

text

Known Issues

Negative indexing not supported

`layout[-1]` raises `IndexError` instead of returning the last element. The binding performs a direct bounds check without normalising negative indices.

Workaround: Use `layout[len(layout) - 1]` or iterate over the layout.

File: `phynexis/bindings/fields/core/pyfield_layout.cpp` **Status:** noted

FieldManager

C++: `phynexis::fields::FieldManager` **Python:** `phynexis.fields.FieldManager` **Header:** `src/fields/core/manager.hpp`

Description

`FieldManager` provides centralized lifecycle management for a collection of fields. It maintains a `KeyedObjectPool` of `FieldBase*` pointers, tracks a `FieldLayout`, and ensures all managed fields share the same size. Use it to create, resize, subset, and persist groups of fields together.

Constructors

FieldManager()

Creates an empty manager with no fields and size 0.

Example:

```

1 import phynexis
2 fm = phynexis.fields.FieldManager()
3 print(fm.field_size(), fm.get_field_count())

```

python

Output:

```

1 0 0

```

text

Methods

create_field(meta)

Creates a single field from metadata and registers it.

Parameters: | Parameter | Type | Description | |-----|----|-----| | meta | FieldMeta | Metadata for the new field |

Returns: FieldHandle — handle to the created field

Example:

```

1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 meta = F.FieldMeta("pressure", F.FieldType.Scalar, F.ValueType.Double)
6 h = fm.create_field(meta)
7 print(h)
8 fm.resize_fields(10)
9 print(fm.field_size())

```

python

Output:

```

1 FieldHandle(index=0, epoch=0)
2 10

```

text

create_fields(metas)

Creates multiple fields from a FieldLayout (or list of FieldMeta).

Parameters: | Parameter | Type | Description | |-----|----|-----| | metas | FieldLayout or list[FieldMeta] | Layout describing the fields to create |

Returns: list[FieldHandle]

Example:

```

1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 layout = F.FieldLayout([
6     F.FieldMeta("x", F.FieldType.Scalar, F.ValueType.Double),
7     F.FieldMeta("v", F.FieldType.VecX, F.ValueType.Double),
8 ])
9 handles = fm.create_fields(layout)
10 print([str(h) for h in handles])

```

python

Output:

```

1 ['FieldHandle(index=0, epoch=0)', 'FieldHandle(index=1, epoch=0)']

```

text

`ensure_fields(layout) / ensure_fields(metas) / ensure_fields(schema)`

Ensures all fields described by the layout/schema exist; creates any that are missing. Existing fields are left untouched.

Parameters: | Parameter | Type | Description | |-----|----|-----| | layout | FieldLayout | Layout to ensure
| metas | list[FieldMeta] | List of metadata (auto-converted to layout) | | schema | FieldSchema | Schema to ensure |

Returns: bool — True if any new field was created

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 fm.create_field(F.FieldMeta("x", F.FieldType.Scalar, F.ValueType.Double))
6 fm.resize_fields(5)
7
8 # "x" already exists, "y" will be created
9 fm.ensure_fields([
10     F.FieldMeta("x", F.FieldType.Scalar, F.ValueType.Double),
11     F.FieldMeta("y", F.FieldType.Scalar, F.ValueType.Double),
12 ])
13 print(fm.get_field_count())
```

python

Output:

```
1 2
```

text

`remove_field(name) / remove_field(handle)`

Removes a field by name or by handle.

Parameters: | Parameter | Type | Description | |-----|----|-----| | name | str | Field name | | handle |
FieldHandle | Field handle |

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 fm.create_field(F.FieldMeta("x", F.FieldType.Scalar, F.ValueType.Double))
6 fm.remove_field("x")
7 print(fm.has_field("x"))
```

python

Output:

```
1 False
```

text

`has_field(name)`

Returns True if a field with the given name exists.

Parameters: | Parameter | Type | Description | |-----|----|-----| | name | str | Field name to check |

Returns: bool

valid_field_handle(handle)

Checks if a handle is still valid (index in range and epoch matches).

Parameters: | Parameter | Type | Description | |-----|----|-----| | handle | FieldHandle | Handle to validate |

Returns: bool

get_field(handle)

Retrieves the concrete field instance for a handle. The runtime type is resolved via RTTI (e.g. `ScalarField`, `VecXdField`).

Parameters: | Parameter | Type | Description | |-----|----|-----| | handle | FieldHandle | Field handle |

Returns: concrete `Field` subclass (e.g. `ScalarField`)

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 h = fm.create_field(F.FieldMeta("p", F.FieldType.Scalar, F.ValueType.Double))
6 fm.resize_fields(5)
7 f = fm.get_field(h)
8 print(type(f).__name__)
```

python

Output:

```
1 ScalarField
```

text

find_field(name)

Finds a field by name and returns its handle.

Parameters: | Parameter | Type | Description | |-----|----|-----| | name | str | Field name |

Returns: FieldHandle

field_size()

Returns the shared element count of all managed fields.

Returns: int

resize_fields(new_size)

Resizes **all** managed fields to `new_size`.

Parameters: | Parameter | Type | Description | |-----|----|-----| | new_size | int | New element count for all fields |

reserve_fields(capacity)

Reserves capacity for all managed fields without changing their sizes.

Parameters: | Parameter | Type | Description | |-----|----|-----| | capacity | int | Capacity to reserve |

clear_fields()

Clears all elements from all fields (size becomes 0). Fields themselves remain registered.

clear()

Removes all fields and resets the manager to its initial empty state.

subset_fields(mask)

Extracts a subset of elements where `mask[i]` is `True`, returning a new `FieldManager`.

Parameters: | Parameter | Type | Description | |----|----|----| | `mask` | `BoolField` | Boolean mask field |

Returns: `FieldManager` — new manager containing the subset

Known Issue: The returned manager may have `field_size() == 0` even when the subset is non-empty. This appears to be a C++ source behavior; the field count and names are correct.

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 fm.create_field(F.FieldMeta("x", F.FieldType.Scalar, F.ValueType.Double))
6 fm.resize_fields(5)
7
8 mask = F.BoolField("mask", 5, True)
9 mask[0] = False
10 mask[2] = False
11
12 fm2 = fm.subset_fields(mask)
13 print(fm2.get_field_count(), fm2.get_field_names())
```

python

Output:

```
1 1 ['x']
```

text

subset_fields_inplace(mask)

Same as `subset_fields` but modifies the current manager in-place.

Parameters: | Parameter | Type | Description | |----|----|----| | `mask` | `BoolField` | Boolean mask field |

check_size_consistency()

Returns `True` if all fields have the same size.

Returns: `bool`

get_inconsistent_fields()

Returns the names of fields whose size differs from the majority.

Returns: `list[str]`

reset_field_values(index)

Resets all field values at `index` to their type defaults.

Parameters: | Parameter | Type | Description | |----|----|----| | `index` | `int` | Element index to reset |

```
copy_field_values(src_index, dst_index)
```

Copies all field values from `src_index` to `dst_index`.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `src_index` | `int` | Source index | | `dst_index` | `int` | Destination index |

```
set_field_value(index, name, value)
```

Sets a single field value by name at the given index.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `index` | `int` | Element index | | `name` | `str` | Field name | | `value` | `float` / `int` / `bool` | Value to set |

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 fm.create_field(F.FieldMeta("p", F.FieldType.Scalar, F.ValueType.Double))
6 fm.resize_fields(3)
7 fm.set_field_value(1, "p", 99.5)
8 ok, val = fm.get_field_value(1, "p")
9 print(val)
```

python

Output:

```
1 {'p': 99.5}
```

text

```
get_field_value(index, name)
```

Reads a single field value by name at the given index.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `index` | `int` | Element index | | `name` | `str` | Field name |

Returns: `tuple[bool, dict]` — (success, {name: value})

```
set_field_values(index, values)
```

Sets multiple field values at an index from a JSON-like dict.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `index` | `int` | Element index | | `values` | `dict` | {field_name: value, ...} |

```
get_field_values(index)
```

Reads all field values at an index as a dict.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `index` | `int` | Element index |

Returns: `dict` — {field_name: value, ...}

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
```

python

```

5 fm.create_field(F.FieldMeta("p", F.FieldType.Scalar, F.ValueType.Double))
6 fm.create_field(F.FieldMeta("t", F.FieldType.Scalar, F.ValueType.Double))
7 fm.resize_fields(2)
8 fm.set_field_value(0, "p", 1.0)
9 fm.set_field_value(0, "t", 300.0)
10 print(fm.get_field_values(0))

```

Output:

```
1 {'p': 1.0, 't': 300.0}
```

text

save_to(path, file, opt=None)

Saves all fields to a binary file.

Parameters: | Parameter | Type | Default | Description | |-----|----|-----|-----| | path | str | — | Parent directory | | file | str | — | Filename | | opt | SaveOptions | SaveOptions() | Save options |

load_from(path, file, opt=None)

Loads fields from a binary file.

Parameters: | Parameter | Type | Default | Description | |-----|----|-----|-----| | path | str | — | Parent directory | | file | str | — | Filename | | opt | LoadOptions | LoadOptions() | Load options |

Example:

```

1 import phynexis, tempfile, os
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 fm.create_field(F.FieldMeta("p", F.FieldType.Scalar, F.ValueType.Double))
6 fm.resize_fields(5)
7
8 with tempfile.TemporaryDirectory() as td:
9     fm.save_to(td, "snapshot.dat")
10    fm2 = F.FieldManager()
11    fm2.load_from(td, "snapshot.dat")
12    print(fm2.get_field_count(), fm2.field_size())

```

python

Output:

```
1 1 5
```

text

inspect(filepath)

Reads metadata from a snapshot file without loading data.

Parameters: | Parameter | Type | Description | |-----|----|-----| | filepath | str | Full path to the snapshot file |

get_field_layout()

Returns the current `FieldLayout` describing all registered fields.

Returns: `FieldLayout`

get_field_count()

Returns the number of registered fields.

Returns: `int`**get_field_names()**

Returns the names of all registered fields.

Returns: `list[str]`**get_field_ptrs()**Returns a list of concrete field instances. Each element is the runtime-resolved type (e.g. `ScalarField`, `VecXdField`).**Returns:** `list[Field]`**print_info()**

Prints a summary of the manager to stdout.

Example:

```

1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 fm.create_field(F.FieldMeta("p", F.FieldType.Scalar, F.ValueType.Double))
6 fm.resize_fields(5)
7 fm.print_info()
```

python

Output:

```

1 === FieldManager ===
2 field count: 1
3 field size: 5
4 - p (double)
```

text

Unexposed C++ API

The following C++ APIs are not exposed via pybind:

- `remove_fields(handles)` / `remove_fields(names)` — batch remove by `VecX<FieldHandle>` or `VecX<String>`

Known Issues

subset_fields returns manager with field_size() == 0

When calling `subset_fields(mask)`, the returned `FieldManager` correctly contains the subset fields (names and count are accurate), but `field_size()` reports 0. This appears to be a C++ source-side behavior rather than a binding issue.

Workaround: Use `subset_fields_inplace(mask)` if in-place mutation is acceptable, or verify field sizes with `get_field_count()` and per-field `size()`.

Status: noted — source issue, not binding bug

FieldMeta

C++: `phynexis::fields::FieldMeta` **Python:** `phynexis.fields.FieldMeta` **Header:** `src/fields/core/field_meta.hpp`

Schema-level metadata describing a field's name, value type, and field type (scalar vs variable-length vector). Used by `FieldSchema` and `FieldManager` for runtime field registration and I/O.

Enums

FieldType

Discriminates scalar fields from variable-length vector fields.

Value	Description
Scalar	One value per grid index
VecX	Variable-length vector per grid index

ValueType

Type tag for scalar values. Maps to phynexis primitive types.

Value	C++ Type	Size
Bool	<code>bool</code>	1 byte
Char	<code>char</code>	1 byte
Float	<code>float</code>	4 bytes
Double	<code>double</code>	8 bytes
Int8	<code>int8_t</code>	1 byte
Int16	<code>int16_t</code>	2 bytes
Int32	<code>int32_t</code>	4 bytes
Int64	<code>int64_t</code>	8 bytes

Constructors

`FieldMeta()`

Default constructor. Creates a scalar double field named "default".

`FieldMeta(name, field_type, value_type)`

Create metadata with explicit parameters.

Parameters:

Parameter	Type	Description
<code>name</code>	<code>str</code>	Field name
<code>field_type</code>	<code>FieldType</code>	<code>FieldType.Scalar</code> or <code>FieldType.VecX</code>
<code>value_type</code>	<code>ValueType</code>	One of the <code>ValueType</code> enum values

Properties

Property	Type	Access	Description
name	str	read/write	Field name
field_type	FieldType	read/write	Scalar or VecX
value_type	ValueType	read/write	Primitive value type

Example

```

1 import phynexis
2
3 # Default construction
4 meta = phynexis.fields.FieldMeta()
5 print("default:", meta.name, meta.field_type, meta.value_type)
6
7 # Explicit construction
8 meta2 = phynexis.fields.FieldMeta(
9     "velocity",
10    phynexis.fields.FieldType.VecX,
11    phynexis.fields.ValueType.Double,
12 )
13 print("explicit:", meta2.name, meta2.field_type, meta2.value_type)
14
15 # Modify properties
16 meta2.name = "pressure"
17 meta2.field_type = phynexis.fields.FieldType.Scalar
18 meta2.value_type = phynexis.fields.ValueType.Float
19 print("modified:", meta2.name, meta2.field_type, meta2.value_type)
20
21 # Enum values
22 print("FieldType:", list(phynexis.fields.FieldType.__members__.keys()))
23 print("ValueType:", list(phynexis.fields.ValueType.__members__.keys()))

```

python

Output:

```

1 default: default FieldType.Scalar ValueType.Double
2 explicit: velocity FieldType.VecX ValueType.Double
3 modified: pressure FieldType.Scalar ValueType.Float
4 FieldType: ['Scalar', 'VecX']
5 ValueType: ['Bool', 'Char', 'Float', 'Double', 'Int8', 'Int16', 'Int32', 'Int64']

```

text

Unexposed C++ API

- `to_string(FieldType) / from_string(String)` — string conversion helpers
- `to_json(json&, FieldType) / from_json(json&, FieldType&)` — JSON serialization
- `to_json(json&, const FieldMeta&) / from_json(json&, FieldMeta&)` — JSON serialization

FieldSchema

C++: `phynexis::fields::FieldSchema` **Python:** `phynexis.fields.FieldSchema` **Header:** `src/fields/core/field_schema.hpp`

Description

`FieldSchema` defines a structured layout of fields. It is a collection of `FieldSlot` objects that describe the names, types, and dimensions of all fields in a data model. A schema can be converted to a `FieldLayout` (used by `FieldManager` to create fields) or used to resolve/normalize JSON dictionaries.

Constructors

`FieldSchema()`

Creates an empty schema with default name "default".

Example:

```
1 import phynexis
2 schema = phynexis.fields.FieldSchema()
3 print(schema.name(), schema.num_slots())
```

python

Output:

```
1 default 0
```

text

Methods

`append(slot) / append(schema)`

Appends a `FieldSlot` or merges another `FieldSchema`. Duplicate slot names are silently skipped.

Parameters: | Parameter | Type | Description | |-----|----|-----| | slot | `FieldSlot` | Slot to append | | schema | `FieldSchema` | Schema whose slots are appended |

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 schema = F.FieldSchema()
5 schema.append(F.FieldSlot.make_scalar("pressure"))
6 schema.append(F.FieldSlot.make_vec3("velocity"))
7 print(schema.num_slots())
8
9 # Merge another schema
10 schema2 = F.FieldSchema()
11 schema2.append(F.FieldSlot.make_scalar("temperature"))
12 schema.append(schema2)
13 print(schema.num_slots())
```

python

Output:

```
1 2
```

text

2 3

name()

Returns the schema name.

Returns: str**num_slots()**

Returns the number of slots.

Returns: int**slots()**

Returns all slots as a list.

Returns: list[FieldSlot]**Example:**

```

1 import phynexis
2 F = phynexis.fields
3
4 schema = F.FieldSchema()
5 schema.append(F.FieldSlot.make_scalar("p"))
6 schema.append(F.FieldSlot.make_vec3("v"))
7 for s in schema.slots():
8     print(s.name, s.dimension)

```

python

Output:

```

1 p 1
2 v 3

```

text

slot(index)

Returns the slot at the given index.

Parameters: | Parameter | Type | Description | |----|----|----| | index | int | Zero-based slot index |**Returns:** FieldSlot**make_layout()**

Converts the schema to a FieldLayout. Vector slots (e.g. vec3) are expanded into individual scalar component entries.

Returns: FieldLayout**Example:**

```

1 import phynexis
2 F = phynexis.fields
3
4 schema = F.FieldSchema()
5 schema.append(F.FieldSlot.make_scalar("pressure"))
6 schema.append(F.FieldSlot.make_vec3("velocity"))
7 schema.append(F.FieldSlot.make_vecx("features"))
8

```

python

```

9 layout = schema.make_layout()
10 print(len(layout))
11 for meta in layout:
12     print(meta.name, meta.field_type, meta.value_type)

```

Output:

```

1 5
2 pressure FieldType.Scalar ValueType.Double
3 velocity.x FieldType.Scalar ValueType.Double
4 velocity.y FieldType.Scalar ValueType.Double
5 velocity.z FieldType.Scalar ValueType.Double
6 features FieldType.VecX ValueType.Double

```

resolve_json(j)

Normalizes a JSON dictionary according to the schema.

- Keys that match schema slots are kept.
- Vector values (lists) are expanded into component keys (`velocity` → `velocity.x`, `velocity.y`, `velocity.z`).
- Extra keys not in the schema are dropped.
- `null` values are skipped.

Parameters: | Parameter | Type | Description | |----|----|-----| | j | dict | Input dictionary |

Returns: dict — normalized dictionary

Example:

```

1 import phynexis
2 F = phynexis.fields
3
4 schema = F.FieldSchema()
5 schema.append(F.FieldSlot.make_scalar("pressure"))
6 schema.append(F.FieldSlot.make_vec3("velocity"))
7
8 j = {
9     "pressure": 1.0,
10    "velocity": [2.0, 3.0, 4.0],
11    "extra": 999, # will be dropped
12 }
13 result = schema.resolve_json(j)
14 print(result)

```

Output:

```
1 {'pressure': 1.0, 'velocity.x': 2.0, 'velocity.y': 3.0, 'velocity.z': 4.0}
```

print_info()

Prints a human-readable summary of the schema to stdout.

Example:

```

1 import phynexis
2 F = phynexis.fields
3
4 schema = F.FieldSchema()
5 schema.append(F.FieldSlot.make_scalar("pressure"))
6 schema.append(F.FieldSlot.make_vec3("velocity"))

```

```
7 schema.print_info()
```

Output:

```
1 FieldSchema: default
2   Slots: 2
3     pressure (scalar, double, 1)
4     velocity (scalar, double, 3)
```

text

Unexposed C++ API

The following C++ APIs are not exposed via pybind:

- Schema construction helpers (`add_scalar`, `add_vec3`, `add_vec4`, `add_vec6`, `add_vecn`, `add_vecx`) — equivalent to `append(FieldSlot.make_*(...))` in Python.
- `to_json` / `from_json` free functions — JSON serialization of the schema itself.

Known Issues

`slots()` returns a snapshot

`schema.slots()` returns a Python list of `FieldSlot` objects. If you call `append()` after `slots()`, the returned list is not automatically updated. Call `slots()` again to get the latest state.

FieldSlot

C++: `phynexis::fields::FieldSlot` **Python:** `phynexis.fields.FieldSlot` **Header:** `src/fields/core/field_slot.hpp`

Description

`FieldSlot` describes a single field entry within a `FieldSchema`. It stores the field's name, type, dimension, and component naming rules. Think of it as a blueprint for one column in a structured data layout.

`FieldSlot` is a plain data struct exposed to Python with read/write attributes.

Constructors

`FieldSlot()`

Creates a default slot with name "default", `FieldType.Scalar`, `ValueType.Double`, and dimension 1.

Example:

```
1 import phynexis
2 slot = phynexis.fields.FieldSlot()
3 print(slot.name, slot.field_type, slot.value_type, slot.dimension)
```

python

Output:

```
1 default FieldType.Scalar ValueType.Double 1
```

text

Factory Methods

Factory methods are the recommended way to create `FieldSlot` instances. They set sensible defaults and generate appropriate component names.

FieldSlot.make_scalar(name, type=ValueType.Double)

Creates a scalar slot.

Parameters: | Parameter | Type | Default | Description | |----|---|----|-----| | name | str | — | Field name | | type | ValueType | ValueType.Double | Value type |

Returns: FieldSlot

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 s = F.FieldSlot.make_scalar("pressure", F.ValueType.Double)
5 print(s.name, s.dimension, s.get_field_names())
```

python

Output:

```
1 pressure 1 ['pressure']
```

text

FieldSlot.make_vec3(name, type=ValueType.Double)

Creates a 3-component vector slot. Component names default to ['x', 'y', 'z'].

Parameters: | Parameter | Type | Default | Description | |----|---|----|-----| | name | str | — | Base name (prefix) | | type | ValueType | ValueType.Double | Value type |

Returns: FieldSlot

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 s = F.FieldSlot.make_vec3("velocity")
5 print(s.get_field_names())
6 print(s.get_component_names())
```

python

Output:

```
1 ['velocity.x', 'velocity.y', 'velocity.z']
2 ['x', 'y', 'z']
```

text

FieldSlot.make_vec4(name, type=ValueType.Double)

Creates a 4-component vector slot. Component names default to ['x', 'y', 'z', 'w'].

Parameters: | Parameter | Type | Default | Description | |----|---|----|-----| | name | str | — | Base name | | type | ValueType | ValueType.Double | Value type |

Returns: FieldSlot

FieldSlot.make_vec6(name, type=ValueType.Double)

Creates a 6-component vector slot. Component names default to ['xx', 'yy', 'zz', 'xy', 'xz', 'yz'].

Parameters: | Parameter | Type | Default | Description | |-----|----|-----|-----| | name | str | — | Base name | | type | ValueType | ValueType.Double | Value type |

Returns: FieldSlot

```
FieldSlot.make_vecn(name, dimension, type=ValueType.Double, delimiter=".",
component_names=[])
```

Creates an N-component vector slot with arbitrary dimension.

Parameters: | Parameter | Type | Default | Description | |-----|----|-----|-----| | name | str | — | Base name | | dimension | int | — | Number of components | | type | ValueType | ValueType.Double | Value type | | delimiter | str | "." | Separator between base name and component | | component_names | list[str] | [] | Custom component names (auto-generated if empty) |

Returns: FieldSlot

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 s = F.FieldSlot.make_vecn("pos", 5, delimiter="_")
5 print(s.get_field_names())
```

python

Output:

```
1 ['pos_c0', 'pos_c1', 'pos_c2', 'pos_c3', 'pos_c4']
```

text

```
FieldSlot.make_vecx(name, type=ValueType.Double)
```

Creates a variable-length vector (VecX) slot.

Parameters: | Parameter | Type | Default | Description | |-----|----|-----|-----| | name | str | — | Field name | | type | ValueType | ValueType.Double | Value type |

Returns: FieldSlot

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 s = F.FieldSlot.make_vecx("features")
5 print(s.name, s.field_type, s.dimension)
```

python

Output:

```
1 features FieldType.VecX 1
```

text

Properties

Property	Type	Access	Description
name	str	read/write	Field name or base name
field_type	FieldType	read-only	Scalar or VecX
value_type	ValueType	read-only	Double, Int32, Bool, etc.
dimension	int	read-only	Number of components (1 for scalar, 3 for vec3, etc.)
delimiter	str	read/write	Separator used in expanded field names
component_names	list[str]	read/write	Component names (e.g. ['x', 'y', 'z'])

Methods

get_field_names()

Returns the fully expanded field names for this slot.

- **Scalar:** returns [name]
- **Vec3:** returns [name.x, name.y, name.z]
- **VecN:** returns [name.c0, name.c1, ...] or custom component names

Returns: list[str]

Example:

```

1 import phynexis
2 F = phynexis.fields
3
4 s1 = F.FieldSlot.make_scalar("p")
5 s2 = F.FieldSlot.make_vec3("v")
6 s3 = F.FieldSlot.make_vecn("c", 2, component_names=["r", "g"])
7
8 print(s1.get_field_names())
9 print(s2.get_field_names())
10 print(s3.get_field_names())

```

python

Output:

```

1 ['p']
2 ['v.x', 'v.y', 'v.z']
3 ['c.r', 'c.g']

```

text

get_component_names()

Returns the component names for this slot.

Returns: list[str]

Example:

```

1 import phynexis
2 F = phynexis.fields
3
4 s = F.FieldSlot.make_vec6("stress")

```

python

```
5 print(s.get_component_names())
```

Output:

```
1 ['xx', 'yy', 'zz', 'xy', 'xz', 'yz']
```

text

Field Views

C++: `phynexis::fields::{FieldViewBase, ScalarFieldView, Vec3FieldView, Vec4FieldView, Vec6FieldView, VecNFieldView, VecXFieldView}` **Python:** `phynexis.fields` **Headers:** `src/fields/views/*.hpp`

Field views are **non-owning wrappers** over existing scalar/vector fields. They provide a unified, type-safe interface for reading and writing field data without copying memory. Views are commonly used for SOA (Structure of Arrays) access patterns — e.g., treating three scalar fields x, y, z as a single vector field.

All view types inherit from `FieldViewBase`.

Type Summary

Python Class	C++ Type	Element / Component Type
<code>FieldViewBase</code>	<code>FieldViewBase</code>	— (abstract)
<code>ScalarFieldView</code>	(alias <code>ScalarFieldView<Double></code>)	float
<code>FieldView</code>		
<code>Int32FieldView</code>	(alias <code>ScalarFieldView<Int32></code>)	int (32-bit)
<code>Idx32FieldView</code>		
<code>Int64FieldView</code>	(alias <code>ScalarFieldView<Int64></code>)	int (64-bit)
<code>Idx64FieldView</code>		
<code>BoolFieldView</code>	<code>ScalarFieldView<bool></code>	bool
<code>Vec3dField</code>	<code>Vec3FieldView<Double></code>	float
<code>Vec3iField</code>	<code>Vec3FieldView<Int32></code>	int (32-bit)
<code>Vec4dField</code>	(alias <code>Vec4FieldView<Double></code>)	float
<code>QuaternionField</code>		
<code>Vec6dField</code>	<code>Vec6FieldView<Double></code>	float
<code>VecNdField</code>	<code>VecNFieldView<Double></code>	float
<code>VecXdFieldView</code>	<code>VecXFieldView<Double></code>	float
<code>VecXiFieldView</code>	<code>VecXFieldView<Int32></code>	int (32-bit)

Common API (FieldViewBase)

All views share the following methods.

`is_valid()`

Returns `True` if the view references valid (non-null) underlying fields.

`__bool__()`

Same as `is_valid()`. Allows `if view:` syntax.

`size() / __len__()`

Returns the number of elements (rows) in the view.

`empty()`

Returns `True` if `size() == 0`.

`get_name(delimiter)`

Returns the base name of the view. For compound views (`Vec3`, `Vec4`, etc.), this strips the component suffix using the given delimiter.

Parameters: | Parameter | Type | Default | Description | |-----|----|-----|-----| | `delimiter` | `str` | `"."` |
| Delimiter used to split component names |

Known issue: The `delimiter` default is not automatically supplied from Python. Pass it explicitly:

```
view.get_name(".").
```

`get_field_names()`

Returns a `list[str]` of the underlying field names.

ScalarFieldView

Single-field view. Wraps one `Field<T>` and provides direct element access.

Constructors

Signature	Description
<code>ScalarFieldView()</code>	Empty (invalid) view.
<code>ScalarFieldView(field)</code>	View over a <code>ScalarField</code> (or <code>FieldBase</code>).

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 h = fm.create_field(F.FieldMeta("temp", F.FieldType.Scalar, F.ValueType.Double))
6 f = fm.get_field(h)
7
8 view = F.ScalarFieldView(f)
9 print(view.size(), view.is_valid()) # 0 True
10
11 # Element access
12 view.resize(3)
13 view[0] = 1.0
14 view[1] = 2.0
15 print(view[0], view[1]) # 1.0 2.0
```

python

`field_ptr()`

Returns the underlying `Field<T>*` pointer.

field_ref()Returns a reference to the underlying field. Same as dereferencing `field_ptr()`.**data()**Returns a pointer to the raw data buffer. **Unsafe** from Python — use `__getitem__` / `__setitem__` instead.`__getitem__(idx)` / `__setitem__(idx, value)`

Direct element read/write. No bounds checking; use with caution.

`__getattr__(name)`

Forwards unknown attribute access to the underlying field object. Allows calling field methods through the view.

Vec3FieldView

View over three scalar fields interpreted as (x, y, z) components.

Constructors

Signature	Description
<code>Vec3dField(x, y, z)</code>	View over three <code>ScalarField</code> (or <code>FieldBase</code>) objects.
<code>Vec3iField(x, y, z)</code>	View over three <code>Int32Field</code> objects.

Example:

```

1  import phynexis
2  F = phynexis.fields
3
4  fm = F.FieldManager()
5  hx = fm.create_field(F.FieldMeta("pos.x", F.FieldType.Scalar, F.ValueType.Double))
6  hy = fm.create_field(F.FieldMeta("pos.y", F.FieldType.Scalar, F.ValueType.Double))
7  hz = fm.create_field(F.FieldMeta("pos.z", F.FieldType.Scalar, F.ValueType.Double))
8
9  x = fm.get_field(hx)
10 y = fm.get_field(hy)
11 z = fm.get_field(hz)
12
13 for f in [x, y, z]:
14     f.resize(3)
15
16 v3 = F.Vec3dField(x, y, z)
17 print(v3.size())           # 3
18 print(v3.get_name("."))    # "pos"
19
20 # Component access (returns ScalarFieldView)
21 print(v3.x().size())       # 3
22
23 # Indexed component access (returns scalar value)
24 print(v3.x(0))              # 0.0 (default-initialized)
25
26 # __getitem__ returns Vec3d (Double version only)
27 v3.x()[0] = 1.0
28 v3.y()[0] = 2.0
29 v3.z()[0] = 3.0
30 print(v3[0])               # Vec3d(1.0, 2.0, 3.0)

```

Component Access

Method	Returns	Description
<code>x()</code>	<code>ScalarFieldView</code>	View over the x-component field
<code>y()</code>	<code>ScalarFieldView</code>	View over the y-component field
<code>z()</code>	<code>ScalarFieldView</code>	View over the z-component field
<code>x(i)</code>	<code>float / int</code>	Value of x at index <code>i</code>
<code>y(i)</code>	<code>float / int</code>	Value of y at index <code>i</code>
<code>z(i)</code>	<code>float / int</code>	Value of z at index <code>i</code>

Vec4FieldView

View over four scalar fields interpreted as `(w, x, y, z)` components. Exposed as `Vec4dField` and aliased as `QuaternionField`.

Constructors

Signature	Description
<code>Vec4dField(w, x, y, z)</code>	View over four <code>ScalarField</code> (or <code>FieldBase</code>) objects.

Component Access

Method	Returns	Description
<code>w()</code>	<code>ScalarFieldView</code>	View over the w-component field
<code>x()</code>	<code>ScalarFieldView</code>	View over the x-component field
<code>y()</code>	<code>ScalarFieldView</code>	View over the y-component field
<code>z()</code>	<code>ScalarFieldView</code>	View over the z-component field
<code>w(i)</code>	<code>float</code>	Value of w at index <code>i</code>
<code>x(i)</code>	<code>float</code>	Value of x at index <code>i</code>
<code>y(i)</code>	<code>float</code>	Value of y at index <code>i</code>
<code>z(i)</code>	<code>float</code>	Value of z at index <code>i</code>

Vec6FieldView

View over six scalar fields interpreted as symmetric tensor components `(xx, yy, zz, xy, xz, yz)`.

Constructors

Signature	Description
<code>Vec6dField(xx, yy, zz, xy, xz, yz)</code>	View over six <code>ScalarField</code> (or <code>FieldBase</code>) objects.

Component Access

Method	Returns	Description
<code>xx()</code>	<code>ScalarFieldView</code>	View over the xx-component field
<code>yy()</code>	<code>ScalarFieldView</code>	View over the yy-component field
<code>zz()</code>	<code>ScalarFieldView</code>	View over the zz-component field
<code>xy()</code>	<code>ScalarFieldView</code>	View over the xy-component field
<code>xz()</code>	<code>ScalarFieldView</code>	View over the xz-component field
<code>yz()</code>	<code>ScalarFieldView</code>	View over the yz-component field

Note: Indexed component access (`xx(i)`, etc.) is **not bound** in Python.

VecNFieldView

View over N scalar fields of arbitrary dimension. Unlike `Vec3/Vec4/Vec6`, the dimension is determined at runtime by the number of fields passed.

Constructors

Signature	Description
<code>VecNdField()</code>	Empty (invalid) view.

Note: Constructors taking a field list (`VecX<FieldBase*>`) are **not bound** in Python because `VecX<FieldBase*>` is not exposed. Create views through `FieldManager` layouts or C++ APIs instead.

`__getitem__(index)`

Returns a `ScalarFieldView` over the field at the given index.

VecXFieldView

View over a `VecXField<T>` (ragged/jagged array field). Provides row-level access.

Constructors

Signature	Description
<code>VecXdFieldView(field)</code>	View over a <code>VecXdField</code> .
<code>VecXiFieldView(field)</code>	View over a <code>VecXiField</code> .

`__call__(i, j)`

Access element at row `i`, column `j`. Returns a mutable reference.

```

1 vxf = F.VecXdField(3)
2 vxf.push_back(0, 1.0)
3 vxf.push_back(0, 2.0)
4
5 view = F.VecXdFieldView(vxf)
6 print(view(0, 0)) # 1.0
7 print(view(0, 1)) # 2.0

```

python

`row_size(i)`

Returns the number of elements in row `i`.

row_capacity(i)Returns the allocated capacity of row *i*.**get_total_elements()**

Returns the total number of stored elements across all rows.

__getitem__(i)Returns a `VecXFieldRowView<T>` for row *i*.

Known Issues

get_name() requires explicit delimiter argument

The C++ default argument `delimiter = "."` is not propagated through pybind11. Always pass a delimiter string:

```
1 name = view.get_name(".") # OK
2 name = view.get_name()   # TypeError
```

python

Status: noted

LinkedFieldRowView.data() may segfault

Iterating a `LinkedFieldRowView` returned from `LinkedFieldView.get_targets()` or `get_sources()` can crash. The underlying C++ `RowView` is a proxy type whose lifetime rules are not fully captured by the Python binding.**Workaround:** Access individual elements by index instead of calling `.data()`.**Status:** noted

VecNFieldView list constructor not exposed

`VecNdField(fields: list[FieldBase])` is not available because `VecX<FieldBase*>` is not registered with pybind11.**Workaround:** Use default constructor; practical construction requires C++-side `BoundLayout` or `FieldManager` integration.**Status:** noted

Vec6FieldView indexed accessors not bound

`xx(i)`, `yy(i)`, etc. are not exposed in Python. Use the component view (`v6.xx()`) and then index the `ScalarFieldView`.**Workaround:**

```
1 v6.xx()[i] # instead of v6.xx(i)
```

python

Status: noted

Field / ScalarField

C++: `phynexis::fields::Field<T>` **Python:** `phynexis.fields.ScalarField / Int32Field / Int64Field /`**BoolField Header:** `src/fields/core/field.hpp`

Concrete one-dimensional homogeneous field container, the workhorse type of the `fields` module. The C++ template `Field<T>` is exposed in Python as one non-templated class per primitive value type. `Field` is an alias for `ScalarField` (i.e. `Field<double>`), and `Idx32Field` / `Idx64Field` are aliases for `Int32Field` / `Int64Field`.

Python class	C++ instantiation	Element type
<code>ScalarField</code> (alias <code>Field</code>)	<code>Field<double></code>	<code>float</code> (64-bit)
<code>Int32Field</code> (alias <code>Idx32Field</code>)	<code>Field<Int32></code>	<code>int</code> (32-bit)
<code>Int64Field</code> (alias <code>Idx64Field</code>)	<code>Field<Int64></code>	<code>int</code> (64-bit)
<code>BoolField</code>	<code>Field<bool></code>	<code>bool</code>

Constructors

Signature	Description
<code>ScalarField()</code>	Empty field, no name.
<code>ScalarField(size)</code>	Default-initialised field of length <code>size</code> .
<code>ScalarField(size, value)</code>	Field of length <code>size</code> filled with <code>value</code> .
<code>ScalarField(name, size=0)</code>	Named field, optionally pre-sized.
<code>ScalarField(name, size, value)</code>	Named field, pre-sized and filled.
<code>ScalarField(data, name)</code>	Build from a <code>VecX<T></code> buffer with a name.
<code>ScalarField(other)</code>	Copy constructor.

The `ScalarField(data: VecX<T>)` and `ScalarField(name, data: VecX<T>)` overloads exist in the binding but require a `VecX<T>` instance, which is not exposed in Python. Use `from_numpy` (see below) for the numpy round-trip instead.

Example:

```
1 import phynexis
2
3 F = phynexis.fields
4
5 print(repr(F.ScalarField()))
6 print(repr(F.ScalarField(5)))
7 print(repr(F.ScalarField(5, 3.14)))
8 print(repr(F.ScalarField("rho", 5)))
9 print(repr(F.ScalarField("rho", 5, 1000.0)))
```

python

Output:

```
1 ScalarField(size=0, name='')
2 ScalarField(size=5, name='')
3 ScalarField(size=5, name='')
4 ScalarField(size=5, name='rho')
5 ScalarField(size=5, name='rho')
```

text

NumPy Interop

`to_numpy()`

Returns a 1-D `numpy.ndarray` with a **copy** of the field data.

Returns: `numpy.ndarray` — same dtype as the field element type.

class method `from_numpy(array)`

Creates a new field whose contents are copied from `array`. The element type is fixed by the binding type, so the dtype must match (`float64` for `ScalarField`, `int32` for `Int32Field`, etc.).

Parameters:

Parameter	Type	Description
<code>array</code>	<code>numpy.ndarray</code>	1-D array; dtype must match the field type

Example:

```

1 import numpy as np
2 import phynexis
3
4 F = phynexis.fields
5
6 sf = F.ScalarField.from_numpy(np.linspace(0.0, 1.0, 5))
7 print(repr(sf))
8 print(sf.to_numpy())

```

python

Output:

```

1 ScalarField(size=5, name='')
2 [0.  0.25 0.5  0.75 1.  ]

```

text

Element Access

Method / operator	Description
<code>field[idx]</code>	Read/write element. No bounds check — out-of-range indices read uninitialised memory.
<code>len(field)</code>	Equivalent to <code>field.size()</code> .
<code>at(index)</code>	Bounds-checked read/write. Raises <code>IndexError</code> if out of range.
<code>at_const(index)</code>	Bounds-checked read-only access.
<code>data()</code>	Returns the value at index 0 (not a buffer). Mostly useful as a presence check.

Example:

```

1 import phynexis
2
3 F = phynexis.fields
4
5 sf = F.ScalarField("v", 4, 0.0)
6 sf[0], sf[1], sf[2], sf[3] = 1.0, 2.0, 3.0, 4.0
7 print("len(sf):", len(sf))
8 print("sf[2]:", sf[2])
9 print("sf.at(2):", sf.at(2))
10
11 try:
12     sf.at(99)
13 except IndexError as e:
14     print("sf.at(99):", repr(e))

```

python

Output:

```

1 len(sf): 4
2 sf[2]: 3.0
3 sf.at(2): IndexError('Field index out of range')

```

`field[idx]` does not validate the index. Negative indices or indices beyond `size()` will return arbitrary memory contents instead of raising. Use `field.at(idx)` whenever the index is not statically known.

Capacity

Method	Description
<code>size()</code>	Number of elements currently stored.
<code>empty()</code>	True if <code>size() == 0</code> .
<code>capacity()</code>	Allocated buffer size; always $\geq$ <code>size()</code> .
<code>resize(new_size)</code>	Grow / shrink to <code>new_size</code> ; new elements are default-initialised.
<code>resize(new_size, value)</code>	Grow to <code>new_size</code> , padding with <code>value</code> .
<code>reserve(capacity)</code>	Pre-allocate to avoid reallocation. Does not change <code>size()</code> .
<code>clear()</code>	Sets <code>size()</code> to 0. Capacity is retained.
<code>push_back(value)</code>	Append a single element.
<code>pop_back()</code>	Remove the last element.

Example:

```

1 import phynexis
2
3 F = phynexis.fields
4
5 sf = F.ScalarField()
6 sf.reserve(100)
7 print("after reserve(100): size=", sf.size(), "capacity=", sf.capacity())
8
9 sf.push_back(1.0)
10 sf.push_back(2.0)
11 print("after 2 push_back:", sf.to_numpy(),
12       "size=", sf.size(), "capacity=", sf.capacity())
13
14 sf.pop_back()
15 print("after pop_back:", sf.to_numpy())
16
17 sf.clear()
18 print("after clear: size=", sf.size(), "capacity=", sf.capacity())
19
20 sf.resize(3)
21 print("resize(3):", sf.to_numpy())
22
23 sf.resize(5, 9.0)
24 print("resize(5, 9.0):", sf.to_numpy())

```

Output:

```

1 after reserve(100): size= 0 capacity= 100
2 after 2 push_back: [1. 2.] size= 2 capacity= 100
3 after pop_back: [1.]
4 after clear: size= 0 capacity= 100

```

```

5  resize(3): [0. 0. 0.]
6  resize(5, 9.0): [0. 0. 0. 9. 9.]

```

Bulk Operations

Method	Description
<code>fill(value)</code>	Set all elements to <code>value</code> .
<code>append(other)</code>	Concatenate <code>other</code> (same field type) onto the end of this field.
<code>append_span(buffer)</code>	Append from a <code>VecX<T></code> buffer (not exposed in Python — see notes below).
<code>swap(other)</code>	Swap contents with another field of the same type, including names.
<code>subset(mask)</code>	Build a <code>FieldHolder</code> containing only elements where <code>mask[i] == True</code> . The mask must be a <code>BoolField</code> of the same length.

Example:

```

1  import numpy as np
2  import phynexis
3
4  F = phynexis.fields
5
6  # fill
7  sf = F.ScalarField("a", 4, 0.0)
8  sf.fill(7.0)
9  print("after fill(7):", sf.to_numpy())
10
11 # append
12 a = F.ScalarField("a", 3, 1.0)
13 b = F.ScalarField("b", 2, 9.0)
14 a.append(b)
15 print("a.append(b):", a.to_numpy())
16
17 # swap (note: name is swapped too)
18 a = F.ScalarField("a", 3, 1.0)
19 b = F.ScalarField("b", 3, 2.0)
20 a.swap(b)
21 print(f"after swap: a name={a.name} data={a.to_numpy()}")
22 print(f"          b name={b.name} data={b.to_numpy()}")
23
24 # subset
25 sf = F.ScalarField.from_numpy(np.array([10.0, 20.0, 30.0, 40.0, 50.0]))
26 mask = F.BoolField.from_numpy(np.array([True, False, True, False, True]))
27 holder = sf.subset(mask)
28 print("subset returns:", type(holder).__name__)

```

Output:

```

1  after fill(7): [7. 7. 7. 7.]
2  a.append(b): [1. 1. 1. 9. 9.]
3  after swap: a name=b data=[2. 2. 2.]
4              b name=a data=[1. 1. 1.]
5  subset returns: FieldHolder

```

Arithmetic (Double only)

ScalarField (i.e. Field<double>) supports element-wise arithmetic with both scalars and other ScalarFields of the same length. **Integer and bool fields do not implement these operators** — only Field<double>.

Operator	Field, Field	Field, scalar	scalar, Field
+	element-wise	broadcast	broadcast
-	element-wise	broadcast	broadcast
*	element-wise	broadcast	broadcast
/	element-wise	broadcast	—

Example:

```
1 import phynexis
2
3 F = phynexis.fields
4
5 a = F.ScalarField("a", 4, 1.0)
6 b = F.ScalarField("b", 4, 2.0)
7
8 print("a + b:", (a + b).to_numpy())
9 print("a - b:", (a - b).to_numpy())
10 print("a * b:", (a * b).to_numpy())
11 print("b / a:", (b / a).to_numpy())
12 print("a + 5:", (a + 5).to_numpy())
13 print("5 + a:", (5 + a).to_numpy())
14 print("5 - a:", (5 - a).to_numpy())
15 print("3 * a:", (3 * a).to_numpy())
```

python

Output:

```
1 a + b: [3. 3. 3. 3.]
2 a - b: [-1. -1. -1. -1.]
3 a * b: [2. 2. 2. 2.]
4 b / a: [2. 2. 2. 2.]
5 a + 5: [6. 6. 6. 6.]
6 5 + a: [6. 6. 6. 6.]
7 5 - a: [4. 4. 4. 4.]
8 3 * a: [3. 3. 3. 3.]
```

text

The mismatched-size case raises `RuntimeError("Field sizes must match")`.

Comparison Semantics

`==` and `!=` test **identity**, not value equality:

```
1 return data_ == other.data_ && name_ == other.name_;
```

cpp

That is, two fields compare equal only if they share the same internal pointer *and* have the same name — effectively only when comparing a field with itself or an alias.

```
1 import phynexis
2
3 F = phynexis.fields
4
5 a = F.ScalarField("x", 3, 1.0)
6 b = F.ScalarField("x", 3, 1.0) # same name and contents, different storage
7 print("identical contents, separate fields a == b:", a == b)
```

python

```
8 print("self-comparison      a == a:", a == a)
```

Output:

```
1 identical contents, separate fields a == b: False
2 self-comparison      a == a: True
```

For **value-equality**, compare `to_numpy()` arrays.

Properties

Property	Type	Access	Description
<code>name</code>	<code>str</code>	read/write	Field name (used by <code>FieldManager</code> , I/O, etc.).

Type Aliases and Integer / Bool Fields

```
1 import phynexis
2
3 F = phynexis.fields
4
5 print("Field is ScalarField:    ", F.Field is F.ScalarField)
6 print("Idx32Field is Int32Field:", F.Idx32Field is F.Int32Field)
7 print("Idx64Field is Int64Field:", F.Idx64Field is F.Int64Field)
8
9 i32 = F.Int32Field("idx", 5, 7)
10 i32[2] = 999
11 print("Int32Field:", i32.to_numpy())
12
13 i64 = F.Int64Field("idx", 3, 10**12)
14 print("Int64Field:", i64.to_numpy())
15
16 bf = F.BoolField("active", 4, True)
17 bf[1] = False
18 print("BoolField :", bf.to_numpy())
```

Output:

```
1 Field is ScalarField:    True
2 Idx32Field is Int32Field: True
3 Idx64Field is Int64Field: True
4 Int32Field: [ 7  7 999  7  7]
5 Int64Field: [1000000000000 1000000000000 1000000000000]
6 BoolField : [ True False  True  True]
```

Known Issues

- **`field[idx]` is unchecked.** Use `field.at(idx)` whenever the index might be out of range or negative — `field[-1]` returns garbage instead of the last element.
- **`data()` returns the first element**, not a buffer pointer. Use `to_numpy()` to obtain the data as an array.
- **`np.array(field, copy=False)` segfaults.** The class declares `pybind11::buffer_protocol()` but no `def_buffer` callback. Use `to_numpy()`.

- **== is identity equality**, not value equality (see above).
- **Field<T> is not registered as a subclass of FieldBase** in pybind, so `isinstance(field, FieldBase)` is `False` and the `FieldBase`-only methods (`ensure_capacity`, `is_same_type`, `erase_indices`, `reorder`, `reset_value`, `swap_elements`, `copy_element`) are unreachable from Python field instances. Tracked in [binding-repair-backlog.md](#) (2026-05-03 entry).

Unexposed C++ API

The following C++ methods exist on `Field<T>` but are not bound:

- `value_type()` / `field_type()` — type tag queries (`ValueType`, `FieldType`).
- `is_same_type(other)` — type-tag equivalent of `isinstance` for fields.
- `erase_indices(indices)`, `reorder(new_to_old)` — bulk index operations.
- `reset_value(i)` / `reset_values()` / `set_value(i, void*)` / `get_value_ptr(i)` — generic value pokes used by `FieldHolder` internals.
- `subset_inplace(mask)` — in-place variant of `subset`.
- `save_to(path, file, opt)` / `load_from(path, file, opt)` — file persistence; the alternative free-function entry points are `write_binary` / `read_binary` (documented separately under field I/O).
- `to_binary(out)` / `from_binary(in)` / `pack(out, idx)` / `unpack(in, idx)` — binary serialisation hooks consumed by the manager.
- `Field<T>(name, comp_id, size, value)` — component-field constructor used by schema-driven layouts.
- Iteration via `begin()` / `end()` — Python-side iteration is available through `to_numpy()` or `[i]` access.

I/O and Utilities

Python: `phynexis.fields` **Headers:** `src/fields/utils/*.hpp`

Free functions for field serialization, MPI communication, and console output. These are utility functions that complement the core field data structures.

Console Output

`field_to_string(field)`

Returns a human-readable string representation of a scalar field.

Supported types: `ScalarField`, `Int32Field`, `Int64Field`

Example:

```
1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 h = fm.create_field(F.FieldMeta("rho", F.FieldType.Scalar, F.ValueType.Double))
```

python

```

6 f = fm.get_field(h)
7 f.resize(3)
8 f[0] = 1.0
9 f[1] = 2.0
10 f[2] = 3.0
11
12 print(F.field_to_string(f))
13 # ' name="rho" size=3 [1, 2, 3]'

```

vecx_field_to_string(field)

Returns a string representation of a VecXdField or VecXiField.

Supported types: VecXdField, VecXiField

Field I/O

VTK I/O

write_vtk(field, path, file)

Writes a scalar field to a VTK file.

Parameters: | Parameter | Type | Description | |-----|-----| | field | ScalarField / Int32Field / Int64Field / BoolField | Field to write | | path | str | Directory path | | file | str | Filename |

read_vtk(field, path, file)

Reads a VTK file into an existing field.

Parameters: | Parameter | Type | Description | |-----|-----| | field | ScalarField / Int32Field / Int64Field / BoolField | Field to populate | | path | str | Directory path | | file | str | Filename |

Example:

```

1 # Write
2 F.write_vtk(f, "/tmp/data", "field.vtk")
3
4 # Read
5 f2 = F.ScalarField()
6 F.read_vtk(f2, "/tmp/data", "field.vtk")

```

python

Binary I/O

write_binary(field, path, file)

Writes a field to a binary file.

read_binary(field, path, file)

Reads a binary file into an existing field.

Example:

```

1 F.write_binary(f, "/tmp/data", "field.bin")
2
3 f2 = F.ScalarField()
4 F.read_binary(f2, "/tmp/data", "field.bin")

```

python

JSON I/O

field_to_json(field)

Converts a field to a Python dict.

Returns: dict with keys name, size, data

Example:

```
1 j = F.field_to_json(f)
2 print(j)
3 # {'name': 'rho', 'size': 3, 'data': [1.0, 2.0, 3.0]}
```

python

field_from_json(field, json)

Populates a field from a Python dict.

Parameters: | Parameter | Type | Description | |-----|----|-----| | field | ScalarField / Int32Field / Int64Field / BoolField | Field to populate | | json | dict | Dictionary with name, size, data keys |

Example:

```
1 f2 = F.ScalarField(3)
2 F.field_from_json(f2, {'name': 'copy', 'size': 3, 'data': [10.0, 20.0, 30.0]})
3 print(f2[0]) # 10.0
```

python

Supported types for all I/O operations: ScalarField, Int32Field, Int64Field, BoolField

Manager I/O

pack_manager(manager, indices)

Serializes a subset of fields from a FieldManager into a bytes buffer.

Parameters: | Parameter | Type | Description | |-----|----|-----| | manager | FieldManager | Source manager | | indices | list[int] | Element indices to pack |

Returns: bytes

unpack_manager(data, manager, target_indices)

Deserializes a bytes buffer into a FieldManager at the specified target indices.

Parameters: | Parameter | Type | Description | |-----|----|-----| | data | bytes | Buffer from pack_manager | | manager | FieldManager | Target manager | | target_indices | list[int] | Target element indices |

Example:

```
1 fm = F.FieldManager()
2 # ... populate fm ...
3
4 # Pack elements 0 and 2
5 blob = F.pack_manager(fm, [0, 2])
6
7 # Unpack into a new manager at indices 10 and 11
8 fm2 = F.FieldManager()
9 F.unpack_manager(blob, fm2, [10, 11])
```

python

VecXField I/O

pack_vecx_field(field, indices)

Packs a subset of elements from a VecXdField or VecXiField into a bytes buffer.

Parameters: | Parameter | Type | Description | |-----|----|-----| | field | VecXdField / VecXiField | Source field | | indices | list[int] | Row indices to pack |

Returns: bytes

`unpack_vecx_field(field, data, target_indices)`

Unpacks a bytes buffer into a `VecXdField` or `VecXiField` at target rows.

Parameters: | Parameter | Type | Description | |-----|----|-----| | field | `VecXdField` / `VecXiField` | Target field | | data | bytes | Buffer from `pack_vecx_field` | | target_indices | `list[int]` | Target row indices |

Example:

```
1 vf = F.VecXdField(5)
2 vf.push_back(0, 1.0)
3 vf.push_back(0, 2.0)
4
5 blob = F.pack_vecx_field(vf, [0])
6
7 vf2 = F.VecXdField(5)
8 F.unpack_vecx_field(vf2, blob, [3])
9 print(vf2.get_row(3)) # [1.0, 2.0]
```

python

MPI Utilities

Note: These functions require `mpi4py` and an MPI environment.

`gather_data(local, global)`

Gathers data from all MPI ranks into a single `global` field on the root rank.

Parameters: | Parameter | Type | Description | |-----|----|-----| | local | `ScalarField` / `Int32Field` | Local data on each rank | | global | `ScalarField` / `Int32Field` | Global buffer on root (pre-sized) |

Supported types: `ScalarField`, `Int32Field`

`scatter_data(global, local, root, counts, displs)`

Scatters data from a `global` field on the root rank to `local` fields on all ranks.

Parameters: | Parameter | Type | Description | |-----|----|-----| | global | `ScalarField` / `Int32Field` | Global data on root | | local | `ScalarField` / `Int32Field` | Local buffer on each rank (pre-sized) | | root | int | Root rank | | counts | `list[int]` | Number of elements per rank | | displs | `list[int]` | Displacement offsets per rank |

Supported types: `ScalarField`, `Int32Field`

Example:

```
1 from mpi4py import MPI
2
3 comm = MPI.COMM_WORLD
4 rank = comm.Get_rank()
5
6 # Gather: each rank has 2 elements
7 local = F.ScalarField(2)
8 local[0] = float(rank * 2)
9 local[1] = float(rank * 2 + 1)
10
11 if rank == 0:
12     global_field = F.ScalarField(comm.Get_size() * 2)
13 else:
14     global_field = F.ScalarField()
```

python

```
15
16 F.gather_data(local, global_field)
```

Known Issues

field_to_json / field_from_json require pybind11_json

If `pybind11_json` is not included in the binding compilation, `field_to_json` raises `TypeError: Unregistered type: nlohmann::json...`

Status: fixed (added `#include <pybind11_json/pybind11_json.hpp>` to `pyfield_io.cpp`)

LinkedFieldView

C++: `phynexis::fields::LinkedFieldView` **Python:** `phynexis.fields.LinkedFieldView` **Header:** `src/fields/views/linked_field_view.hpp`

Bidirectional many-to-many link view over two `VecXField<Int64>` storages (`a_to_b` and `b_to_a`). Each row stores a list of linked indices. `LinkedFieldView` itself does not allocate index storage; it borrows the two `VecXField` objects and provides thread-safe `link()` / `unlink()` operations.

`LinkedFieldView` also supports **attached data**: `VecXField<T>` objects can be registered via `attach()`. When a link is created or removed, the attached fields are updated in sync (append on link, swap-remove on unlink).

Related Types

Python Class	C++ Type	Description
<code>LinkedFieldView</code>	<code>LinkedFieldView</code>	Bidirectional link manager
<code>LinkedFieldRowView</code>	<code>LinkedFieldRowView</code>	Read-only view of one link row
<code>ILinkedFieldAttachedData</code>	<code>ILinkedFieldAttachedData</code>	Abstract base for attached data adapters
<code>VecXFieldAttachedAdapterDouble</code>	<code>VecXFieldAttachedAdapter<Double></code>	Adapter for <code>VecXdField</code> attached data
<code>VecXFieldAttachedAdapterInt32</code>	<code>VecXFieldAttachedAdapter<Int32></code>	Adapter for <code>VecXiField</code> attached data

LinkedFieldView

Constructors

Signature	Description
<code>LinkedFieldView()</code>	Empty (invalid) view.
<code>LinkedFieldView(a_to_b, b_to_a)</code>	Links over two <code>VecXlField</code> instances.

The constructor takes **pointers** (kept alive by pybind11) and stores them internally.

Example:

```

1  import phynexis
2  F = phynexis.fields
3
4  # Two VecXlField storages: 3 targets (A) x 3 sources (B)
5  a2b = F.VecXlField(3)
6  b2a = F.VecXlField(3)
7
8  lfv = F.LinkedFieldView(a2b, b2a)
9  print(lfv.size_targets(), lfv.size_sources()) # 3 3
10
11 # Create links
12 lfv.link(0, 1)
13 lfv.link(0, 2)
14 lfv.link(1, 0)
15
16 # Query targets of A=0
17 row = lfv.get_targets(0)
18 print(row.size()) # 2
19 print(row[0], row[1]) # 1 2
20
21 # Query sources of B=1
22 row2 = lfv.get_sources(1)
23 print(row2[0]) # 0

```

`link(a, b)`

Creates a bidirectional link between `a` (target index) and `b` (source index). Thread-safe.

`unlink(a, b)`

Removes the bidirectional link between `a` and `b`. Uses swap-remove on the internal rows, so link order is not preserved. Thread-safe.

`clear_links()`

Removes all links and clears attached data rows.

`size_targets()` / `size_sources()`

Returns the number of rows in `a_to_b` / `b_to_a` respectively.

`get_targets(a)` / `get_sources(b)`

Returns a `LinkedFieldRowView` containing the linked indices for target `a` / source `b`.

`a_to_b_ptr()` / `b_to_a_ptr()`

Returns the underlying `VecXlField` pointers.

attach(field)

Attaches a `VecXdField` or `VecXiField` to be kept in sync with links. On `link(a, b)`, a new element is appended to row `a` of the attached field. On `unlink(a, b)`, the element at the same index is swap-removed.

Supported field types: `VecXdField`, `VecXiField`

attached_count()

Returns the number of attached data fields.

LinkedFieldRowView

Read-only view of a single row in a `LinkedFieldView`. Created by `get_targets()` or `get_sources()`.

size() / __len__()

Returns the number of linked indices in this row.

__getitem__(i)

Returns the linked index at position `i`.

data()

Returns the raw row data as a Python `list`. **Known issue:** This method may segfault due to lifetime issues with the underlying proxy pointer.

Workaround: Iterate using `__getitem__` or `for i in range(row.size()): row[i]`.

Attached Data Adapters

`ILinkedFieldAttachedData` is the abstract base class for attached data. Two concrete adapters are exposed:

VecXFieldAttachedAdapterDouble(field)

Wraps a `VecXdField` for use as attached data. The field must have the same row layout as the `a_to_b` storage.

VecXFieldAttachedAdapterInt32(field)

Wraps a `VecXiField` for use as attached data.

Example:

```
1 # Create a VecXdField with matching row layout
2 weights = F.VecXdField(3)
3 weights.push_back(0, 1.0)
4 weights.push_back(1, 2.0)
5
6 # Attach to LinkedFieldView
7 lfv.attach(weights)
8 print(lfv.attached_count()) # 1
9
10 # Now link() will append a default value to row 'a' of weights
11 lfv.link(0, 2)
```

python

Known Issues

`LinkedFieldRowView.data()` segfault

Calling `.data()` on a `LinkedFieldRowView` may crash. The underlying C++ `RowView` holds a raw pointer to `VecXField` row data; the Python binding does not extend the parent `LinkedFieldView` lifetime.

Workaround: Access elements by index.

Status: noted

`LinkedFieldView` default constructor not bound

`LinkedFieldView()` (no-arg default constructor) is **not exposed** in Python.

Workaround: Always construct with two `VecXlField` arguments.

Status: noted

`get_name()` requires explicit delimiter

Same as other field views: `view.get_name(".")` works; `view.get_name()` raises `TypeError`.

Status: noted

Operators

Python: `phynexis.fields` **Headers:** `src/fields/operators/*.hpp`

Free functions for element-wise math, reductions, prefix sums, and sorting on `Field` objects. All functions operate on `phynexis.fields` types and return new fields or scalar values.

Math

Element-wise mathematical operations. All functions accept `ScalarField` and return a new `ScalarField` of the same size.

Function	Signature	Description
<code>sqrt</code>	<code>sqrt(field)</code>	Square root
<code>sqr</code>	<code>sqr(field)</code>	Square ($x * x$)
<code>sin</code>	<code>sin(field)</code>	Sine
<code>cos</code>	<code>cos(field)</code>	Cosine
<code>exp</code>	<code>exp(field)</code>	Exponential (e^x)
<code>abs</code>	<code>abs(field)</code>	Absolute value
<code>log</code>	<code>log(field)</code>	Natural logarithm
<code>pow</code>	<code>pow(field, exponent)</code>	Power ($x ** exponent$)
<code>tan</code>	<code>tan(field)</code>	Tangent
<code>floor</code>	<code>floor(field)</code>	Floor
<code>ceil</code>	<code>ceil(field)</code>	Ceiling
<code>round</code>	<code>round(field)</code>	Round to nearest integer
<code>fabs</code>	<code>fabs(field)</code>	Floating-point absolute value
<code>atan</code>	<code>atan(field)</code>	Arc tangent
<code>sinh</code>	<code>sinh(field)</code>	Hyperbolic sine
<code>max</code>	<code>max(a, b) / max(field, value)</code>	Element-wise maximum
<code>min</code>	<code>min(a, b) / min(field, value)</code>	Element-wise minimum

Example:

```

1 import phynexis
2 F = phynexis.fields
3
4 fm = F.FieldManager()
5 h = fm.create_field(F.FieldMeta("x", F.FieldType.Scalar, F.ValueType.Double))
6 f = fm.get_field(h)
7 f.resize(3)
8 f[0] = 1.0
9 f[1] = 4.0
10 f[2] = 9.0
11
12 print(F.sqrt(f)[0]) # 1.0
13 print(F.sqrt(f)[1]) # 2.0
14 print(F.sqr(f)[0]) # 1.0
15 print(F.pow(f, 0.5)[1]) # 2.0

```

python

Factory Functions

Function	Signature	Description
<code>zeros</code>	<code>zeros(size)</code>	ScalarField of zeros
<code>ones</code>	<code>ones(size)</code>	ScalarField of ones
<code>linspace</code>	<code>linspace(start, end, num)</code>	<code>num</code> evenly spaced values
<code>arange</code>	<code>arange(start, end, step=1)</code>	Range with step
<code>fill</code>	<code>fill(field, value)</code>	Fill field with a scalar value

Example:

```

1 z = F.zeros(5)
2 o = F.ones(3)
3 ls = F.linspace(0.0, 1.0, 5) # [0.0, 0.25, 0.5, 0.75, 1.0]

```

python

```
4 ar = F.arange(0.0, 5.0, 1.0) # [0.0, 1.0, 2.0, 3.0, 4.0]
```

Vector Operations

dot(a, b)

Element-wise dot product of two `Vec3dField` objects. Returns a `ScalarField`.

Example:

```
1 a = F.Vec3dField(ax, ay, az)
2 b = F.Vec3dField(bx, by, bz)
3 d = F.dot(a, b) # ScalarField of a[i] . b[i]
```

python

cross(a, b)

Element-wise cross product of two `Vec3dField` objects.

Returns: `_Vec3dFieldCrossResult` — call `.get()` to obtain the `Vec3dField`.

Example:

```
1 c = F.cross(a, b)
2 cv = c.get()
3 print(cv.x(0), cv.y(0), cv.z(0))
```

python

Known issue: Calling `.get()` may produce a segfault in some cases due to lifetime issues with the internal `shared_ptr` fields.

Reduction

Aggregate operations that return a single scalar value.

Function	Signature	Description
<code>sum</code>	<code>sum(field)</code>	Sum of all elements
<code>mean</code>	<code>mean(field)</code>	Arithmetic mean
<code>variance</code>	<code>variance(field)</code>	Population variance
<code>std_dev</code>	<code>std_dev(field)</code>	Standard deviation
<code>max_element</code>	<code>max_element(field)</code>	Maximum value
<code>min_element</code>	<code>min_element(field)</code>	Minimum value
<code>any</code>	<code>any(bool_field)</code>	True if any element is True
<code>all</code>	<code>all(bool_field)</code>	True if all elements are True
<code>count</code>	<code>count(bool_field)</code>	Count of True elements

Example:

```
1 print(F.sum(f)) # 55.0
2 print(F.mean(f)) # 11.0
3 print(F.max_element(f)) # 25.0
4 print(F.min_element(f)) # 1.0
```

python

Prefix Sum

Cumulative sum operations.

PrefixSumMode enum

Value	Description
PrefixSumMode.Inclusive	Include current element in sum
PrefixSumMode.Exclusive	Exclude current element in sum

Functions

Function	Signature	Description
<code>prefix_sum_inclusive</code>	<code>prefix_sum_inclusive(field)</code>	Inclusive scan (returns new field)
<code>prefix_sum_exclusive</code>	<code>prefix_sum_exclusive(field)</code>	Exclusive scan (returns new field)
<code>prefix_sum</code>	<code>prefix_sum(field, mode=Inclusive)</code>	Generic scan (returns new field)
<code>prefix_sum_serial</code>	<code>prefix_sum_serial(field, mode=Inclusive)</code>	Single-threaded scan
<code>prefix_sum_inplace_inclusive</code>	<code>prefix_sum_inplace_inclusive(field)</code>	In-place inclusive scan
<code>prefix_sum_inplace_exclusive</code>	<code>prefix_sum_inplace_exclusive(field)</code>	In-place exclusive scan
<code>prefix_sum_inplace</code>	<code>prefix_sum_inplace(field, mode=Inclusive)</code>	Generic in-place scan

Example:

```

1 f.resize(5)
2 for i in range(5):
3     f[i] = float(i + 1) # [1, 2, 3, 4, 5]
4
5 ps = F.prefix_sum_inclusive(f)
6 print([ps[i] for i in range(5)]) # [1, 3, 6, 10, 15]
7
8 ps2 = F.prefix_sum_exclusive(f)
9 print([ps2[i] for i in range(5)]) # [0, 1, 3, 6, 10]
```

python

Sort**sort(field)**

In-place sort of a `ScalarField` in ascending order.

sort(keys, values)

Sort `keys` and reorder `values` accordingly (key-value sort).

sort_by_value(keys, values)

Sort `keys` by the corresponding `values` (indirect sort).

Supported types: `ScalarField`, `Int32Field`, `Int64Field` for both `keys` and `values`.

Example:

```

1 k = F.Int32Field(5)
2 v = F.ScalarField(5)
3 for i in range(5):
4     k[i] = 4 - i
5     v[i] = float(i)
6
7 F.sort_by_value(k, v)
```

python

```
8 print([k[i] for i in range(5)]) # [0, 1, 2, 3, 4]
```

Radix Sort

High-performance integer sort using radix sort algorithm.

radix_sort(keys, values)

Sort integer keys and reorder values accordingly.

radix_sort_by_value(keys, values)

Sort keys by integer values.

Supported types: Int32Field, Int64Field for both keys and values.

Example:

```
1 k = F.Int32Field(5)
2 v = F.Int32Field(5)
3 for i in range(5):
4     k[i] = 4 - i
5     v[i] = i
6
7 F.radix_sort(k, v)
8 print([k[i] for i in range(5)]) # [0, 1, 2, 3, 4]
```

python

Known Issues

cross().get() may segfault

The `cross()` function returns a `_Vec3dFieldCrossResult` object. Calling `.get()` on it can crash in some environments due to `shared_ptr` lifetime management between pybind11 and the internal `Vec3Field` view.

Workaround: Use manual component-wise cross product if stability is required.

Status: noted

phynexis.parsim

Python: phynexis.parsim **pybind module:** pyparsim **Header:** src/parsim/parsim.hpp

Particle simulation framework. Provides node/edge/hyperedge graph structures, spatial indexing, operator-based simulation loop, and model factories for contact mechanics.

Module Status

	Class / Group	Status	Doc
	Schemas & BoundLayouts	done	schemas
Graph	(NodeSet, EdgeSet, HyperEdgeSet, ComputationalGraph)	done	graph
	Simulator, Context, OperatorSystem	done	simulator
	Models & Utils	done	models-utils

Submodules

- **schema** — NodeKinematicsSchema, NodeShapeSchema, NodeInertiaSchema, NodeBoundsSchema, NodeNodeLinkSchema, NodeGridLinkSchema, NodeDomainLinkSchema
- **graph** — ComputationalGraph, NodeSet, EdgeSet, HyperEdgeSet, SetBase, ShapeStore
- **simulator** — Simulator, Context, OperatorSystem, RuntimeState, DomainSystem
- **models** — RepulsionModel, LinearRepulsionModel, RepulsionModelFactory
- **utils** — NodeGenerator, FieldSampler, SpatialIndex, UniformGridIndex
- **operators** — phases, integration, forces, interaction, output submodules

Quick Start

```

1 import phynexis
2 pm = phynexis.parsim
3
4 # Create a graph with nodes
5 g = pm.ComputationalGraph()
6 ns = g.nodes()
7
8 # Add a node (requires schema + bound layout)
9 schema = pm.NodeKinematicsSchema()
10 layout = pm.BoundLayoutNodeKinematicsSchema.make_bound(ns.field_manager(), schema)
11 ns.set_view(layout)
12
13 # Initialize node storage
14 ns.initialize()
15 print(ns.size()) # 0

```

python

See also

- [Python API overview](#)
- [fields module](#) — field data structures used by parsim

Graph

Python: `phynexis.parsim` **Headers:** `src/parsim/graph/*.hpp`

Graph data structures for particle simulations. `ComputationalGraph` is the top-level container holding `NodeSet`, `EdgeSet`, `HyperEdgeSet`, and `ShapeStore`.

ComputationalGraph

Top-level graph container.

Constructors

Signature	Description
<code>ComputationalGraph()</code>	Empty graph.

Methods

Method	Returns	Description
<code>nodes()</code>	<code>NodeSet</code>	The node set
<code>edges()</code>	<code>EdgeSet</code>	The edge set
<code>hyper_edges()</code>	<code>HyperEdgeSet</code>	The hyperedge set
<code>shape_store()</code>	<code>ShapeStore</code>	The shape store
<code>node_count()</code>	<code>int</code>	Number of nodes
<code>edge_count()</code>	<code>int</code>	Number of edges
<code>hyper_edge_count()</code>	<code>int</code>	Number of hyperedges
<code>clear()</code>	—	Clear all sets
<code>is_empty()</code>	<code>bool</code>	True if all sets are empty
<code>save_to(path, file, opt)</code>	—	Save graph to file
<code>load_from(path, file, opt)</code>	—	Load graph from file
<code>inspect(filepath)</code>	—	Inspect file metadata

Example:

```

1 import phynexis
2 pm = phynexis.parsim
3
4 g = pm.ComputationalGraph()
5 print(g.is_empty()) # True
6
7 ns = g.nodes()
8 print(type(ns).__name__) # NodeSet

```

python

SetBase

Abstract base class for `NodeSet`, `EdgeSet`, `HyperEdgeSet`.

Methods

Method	Returns	Description
<code>ensure_fields()</code>	—	Ensure all schema fields exist
<code>schema()</code>	<code>FieldSchema</code>	The set's schema
<code>field_manager()</code>	<code>FieldManager</code>	Underlying field manager
<code>set_view(layout)</code>	—	Bind a <code>BoundLayout</code> to this set
<code>count()</code> / <code>size()</code>	<code>int</code>	Number of elements
<code>empty()</code>	<code>bool</code>	True if size == 0
<code>clear()</code>	—	Remove all elements
<code>emplace()</code>	<code>int</code>	Add a new element (returns ID)
<code>emplace_with_id(id)</code>	—	Add a new element with given ID
<code>erase(id)</code>	—	Remove element by ID
<code>allocate_id()</code>	<code>int</code>	Allocate a new unique ID
<code>save_to(path, file, opt)</code>	—	Save set data
<code>load_from(path, file, opt)</code>	—	Load set data
<code>inspect(filepath)</code>	—	Inspect file metadata

NodeSet

Particle/node storage.

Constructors

Signature	Description
<code>NodeSet()</code>	Empty node set.

Methods

Method	Description
<code>add_node(id, data)</code>	Add a node with JSON data
<code>add_nodes(data_list)</code>	Batch add nodes from list of JSON
<code>remove_node(id)</code>	Remove node by ID
<code>remove_nodes(ids)</code>	Batch remove nodes
<code>has_node(id)</code>	Check if node exists
<code>initialize()</code>	Initialize internal storage
<code>print_info()</code>	Print debug info
<code>inspect_fields()</code>	Inspect field layout
<code>views()</code>	Return node views

Example:

```

1 import phynexis
2 pm = phynexis.parsim
3 F = phynexis.fields
4
5 ns = pm.NodeSet()
6
7 # Bind a schema layout

```

python

```

8 fm = F.FieldManager()
9 schema = pm.NodeKinematicsSchema()
10 layout = pm.BoundLayoutNodeKinematicsSchema.make_bound(fm, schema)
11 ns.set_view(layout)
12 ns.initialize()
13
14 # Add nodes
15 ns.add_node(0, {"pos": [0.0, 0.0, 0.0]})
16 ns.add_node(1, {"pos": [1.0, 0.0, 0.0]})
17 print(ns.size()) # 2

```

EdgeSet

Pairwise edge/link storage.

Constructors

Signature	Description
<code>EdgeSet()</code>	Empty edge set.

Methods

Method	Description
<code>initialize()</code>	Initialize internal storage
<code>print_info()</code>	Print debug info
<code>views()</code>	Return edge views

Note: No direct `add_edge` / `remove_edge` methods are exposed. Use `SetBase.emplace()` / `erase()` for bulk operations.

HyperEdgeSet

Many-to-many hyperedge storage.

Constructors

Signature	Description
<code>HyperEdgeSet()</code>	Empty hyperedge set.

Methods

Method	Description
<code>initialize()</code>	Initialize internal storage
<code>print_info()</code>	Print debug info
<code>views()</code>	Return hyperedge views

ShapeStore

Stores geometric shapes with stable handles.

Constructors

Signature	Description
<code>ShapeStore()</code>	Empty store.

Methods

Method	Returns	Description
<code>insert(shape)</code>	<code>int</code>	Insert shape clone, returns auto-allocated ID
<code>insert_with_id(id, shape)</code>	<code>int</code>	Insert with specified ID
<code>id_at(handle)</code>	<code>int</code>	Get ID from handle (-1 if invalid)
<code>find(id)</code>	<code>ShapeStoreHandle</code>	Find handle by ID
<code>get(handle)</code>	<code>ShapeStoreEntry</code>	Get entry by handle
<code>erase(id)</code>	—	Erase shape by ID
<code>contains(id)</code>	<code>bool</code>	Check if ID exists
<code>valid(handle)</code>	<code>bool</code>	Check if handle is valid
<code>set_label(id, label)</code>	—	Set label
<code>set_need_sync(id, need_sync)</code>	—	Set sync flag
<code>get_label_of(id)</code>	<code>str</code>	Get label
<code>initialize_mpi()</code>	—	Initialize MPI support
<code>sync()</code>	—	Sync across MPI ranks
<code>get_shape_ids()</code>	<code>list[int]</code>	All shape IDs
<code>size()</code>	<code>int</code>	Number of shapes
<code>to_json()</code>	<code>dict</code>	Export all shapes as JSON
<code>save_to(path, file, opt)</code>	—	Save to file
<code>load_from(path, file, opt)</code>	—	Load from file
<code>inspect(filepath)</code>	—	Inspect file metadata

Example:

```

1 import phynexis
2 pm = phynexis.parsim
3 U = phynexis.utils
4
5 ss = pm.ShapeStore()
6 sphere = U.shape.Sphere(0.5)
7 sid = ss.insert(sphere)
8 print(sid) # 0
9
10 entry = ss.get(ss.find(sid))
11 print(entry.label) # ''
12
13 ss.set_label(sid, "particle")
14 print(ss.get_label_of(sid)) # "particle"

```

python

ShapeStoreEntry

Read-only entry returned by `ShapeStore.get()`.

Attribute	Type	Description
<code>shape</code>	<code>Shape</code>	The stored shape
<code>label</code>	<code>str</code>	User-defined label
<code>need_sync</code>	<code>bool</code>	MPI sync flag

ShapeStoreHandle

Lightweight handle with `index` and `epoch` fields.

Views

NodeViewById / NodeViewByIndex

Class	Description
<code>NodeViewById</code>	View nodes by stable ID
<code>NodeViewByIndex</code>	View nodes by contiguous index

EdgeViewById / EdgeViewByIndex

Class	Description
<code>EdgeViewById</code>	View edges by stable ID
<code>EdgeViewByIndex</code>	View edges by contiguous index

HyperEdgeViewById / HyperEdgeViewByIndex

Class	Description
<code>HyperEdgeViewById</code>	View hyperedges by stable ID
<code>HyperEdgeViewByIndex</code>	View hyperedges by contiguous index

PropertiesView

Generic property accessor view for nodes/edges.

Known Issues

EdgeSet / HyperEdgeSet lack add/remove convenience methods

Only `initialize()`, `print_info()`, and `views()` are exposed. Use `SetBase.emplace()` / `erase()` for modifications.

Status: noted

ShapeStoreHandle uses `module_local()`

`ShapeStoreHandle` is registered with `pybind11::module_local()` to avoid conflict with `FieldHandle` from `phynexis.fields`. Cross-module type checking (e.g., `isinstance(handle, FieldHandle)`) will not work.

Status: noted

Models and Utils

Python: `phynexis.parsim` **Headers:** `src/parsim/models/*.hpp, src/parsim/utils/*.hpp`

Contact models, node generation utilities, and spatial indexing helpers.

RepulsionModel

Base class for contact repulsion models.

Methods

Method	Returns	Description
<code>name()</code>	<code>str</code>	Model name
<code>clone()</code>	<code>RepulsionModel*</code>	Deep copy

LinearRepulsionModel

Simple linear spring repulsion.

Constructors

Signature	Description
<code>LinearRepulsionModel()</code>	Default stiffness.
<code>LinearRepulsionModel(stiffness)</code>	Specify stiffness.

Methods

Method	Returns	Description
<code>stiffness()</code>	<code>float</code>	Current stiffness
<code>set_stiffness(v)</code>	—	Set stiffness

Example:

```

1 import phynexis
2 pm = phynexis.parsim
3
4 model = pm.LinearRepulsionModel(1000.0)
5 print(model.stiffness()) # 1000.0
6 print(model.name())     # "LinearRepulsionModel"
7
8 clone = model.clone()
9 print(type(clone).__name__) # LinearRepulsionModel

```

python

RepulsionModelFactory

Factory for creating repulsion models by name.

Static Methods

Method	Returns	Description
<code>create_model(name)</code>	<code>RepulsionModel*</code>	Create model by name
<code>get_available_models()</code>	<code>list[str]</code>	List registered model names
<code>is_registered(name)</code>	<code>bool</code>	Check if name is registered
<code>print_info()</code>	—	Print factory info

Example:

```
1 print(pm.RepulsionModelFactory.get_available_models())
2 model = pm.RepulsionModelFactory.create_model("LinearRepulsionModel")
```

python

NodeGenerator

Generates node property sets from field samplers.

Constructors

Signature	Description
<code>NodeGenerator()</code>	Empty generator.

Methods

Method	Description
<code>add_field(slot, sampler)</code>	Add a field slot with sampler
<code>add_field(name, sampler)</code>	Add Vec3 field by name (spatial sampler)
<code>add_field(name, dist)</code>	Add scalar field by name (distribution)
<code>add_field(name, generator)</code>	Add field by callable (double/Vec3/JSON)
<code>erase_field(name)</code>	Remove a field slot
<code>contains(name)</code>	Check if slot exists
<code>slots()</code>	Get all slots
<code>num_slots()</code>	Number of slots
<code>slot(i) / slot(name)</code>	Get slot by index or name
<code>generate_one()</code>	Generate one node's properties (JSON)
<code>generate(count)</code>	Generate <code>count</code> nodes (list of JSON)
<code>clear()</code>	Remove all slots
<code>print_slots()</code>	Print slot info to stderr

Example:

```
1 import phynexis
2 pm = phynexis.parsim
3 U = phynexis.utils
4
5 gen = pm.NodeGenerator()
6
7 # Add a scalar field with fixed value
8 gen.add_field("radius", lambda: 0.5)
9
10 # Add a Vec3 field with random position
11 sampler = U.sampling.spatial.UniformBoxSampler(
```

python

```

12     U.Vec3d(0, 0, 0), U.Vec3d(1, 1, 1)
13 )
14 gen.add_field("pos", sampler)
15
16 # Generate 3 nodes
17 nodes = gen.generate(3)
18 print(len(nodes)) # 3
19 print(nodes[0])   # {'radius': 0.5, 'pos': [x, y, z]}

```

FieldSampler

Factory for creating field samplers used by `NodeGenerator`.

Static Methods

Method	Returns	Description
<code>make_scalar(dist)</code>	<code>FieldSampler</code>	From distribution
<code>make_scalar(generator)</code>	<code>FieldSampler</code>	From callable () -> double
<code>make_scalar(value)</code>	<code>FieldSampler</code>	From constant value
<code>make_vec3(sampler)</code>	<code>FieldSampler</code>	From spatial sampler
<code>make_vec3(generator)</code>	<code>FieldSampler</code>	From callable () -> <code>Vec3d</code>
<code>make_vec4(sampler)</code>	<code>FieldSampler</code>	From orientation sampler
<code>make_json(generator)</code>	<code>FieldSampler</code>	From callable () -> dict

Node Property Utils

Free functions for node property manipulation.

`apply_density(view, density)`

Applies density to node mass/inertia fields.

`assign_shape(view, shape_id)`

Assigns a shape ID to nodes.

`update_node_bounds(nodes, skin_factor=0.05, margin_factor=0.1)`

Updates AABB bounds for all nodes.

Example:

```

1 ns = pm.NodeSet()
2 # ... setup and initialize ...
3
4 pm.update_node_bounds(ns, 0.05, 0.1)

```

python

SpatialIndex / UniformGridIndex

See [simulator](#) for full documentation.

Quick example:

```

1 idx = pm.UniformGridIndex()
2 config = idx.config()

```

python

```

3 config.min_bound = U.Vec3d(0, 0, 0)
4 config.max_bound = U.Vec3d(10, 10, 10)
5 config.grid_size = U.Vec3d(1, 1, 1)
6
7 idx.build(ns)
8 result = idx.query(U.Vec3d(5, 5, 5), 2.0)
9 print(result) # list of node indices within radius 2.0

```

Known Issues

`NodeGenerator.generate()` returns `list[dict]` not `VecX<json>`

The binding wraps `VecX<json>` to `py::list` for compatibility.

Status: noted

Schemas

Python: `phynexis.parsim` **Headers:** `src/parsim/schema/*.hpp`

Schema definitions for particle node properties. Each schema describes a set of field slots (position, velocity, radius, etc.) that are allocated in a `FieldManager` and bound to a `NodeSet` via a `BoundLayout`.

All schemas inherit from `phynexis.fields.FieldSchema`.

Schema Types

Python Class	Description	Slots
<code>NodeKinematicsSchema</code>	Position, velocity, acceleration, orientation	<code>pos, vel, acc, quat</code>
<code>NodeShapeSchema</code>	Shape ID and radius	<code>shape_id, radius</code>
<code>NodeInertiaSchema</code>	Mass and inertia tensor	<code>mass, inertia</code>
<code>NodeBoundsSchema</code>	AABB bounding box	<code>min_bound,</code> <code>max_bound</code>
<code>NodeNodeLinkSchema</code>	Node-to-node connectivity	<code>target, link_type</code>
<code>NodeGridLinkSchema</code>	Grid-cell linkage	<code>cell_id,</code> <code>grid_level</code>
<code>NodeDomainLinkSchema</code>	MPI domain linkage	<code>domain_id, rank</code>

Constructors

All schemas have a default constructor:

```
1 schema = pm.NodeKinematicsSchema()
```

python

with_prefix(prefix) (NodeNodeLinkSchema only)

Returns a copy of the schema with the given slot name prefix. Static method.

```
1 schema = pm.NodeNodeLinkSchema.with_prefix("bond_")
```

python

BoundLayout

A `BoundLayout` binds a schema to a concrete `FieldManager`, allocating the actual fields.

Python Class	Schema
<code>BoundLayoutNodeKinematicsSchema</code>	<code>NodeKinematicsSchema</code>
<code>BoundLayoutNodeShapeSchema</code>	<code>NodeShapeSchema</code>
<code>BoundLayoutNodeInertiaSchema</code>	<code>NodeInertiaSchema</code>
<code>BoundLayoutNodeBoundsSchema</code>	<code>NodeBoundsSchema</code>
<code>BoundLayoutNodeNodeLinkSchema</code>	<code>NodeNodeLinkSchema</code>
<code>BoundLayoutNodeGridLinkSchema</code>	<code>NodeGridLinkSchema</code>
<code>BoundLayoutNodeDomainLinkSchema</code>	<code>NodeDomainLinkSchema</code>
<code>BoundLayoutSetSchema</code>	Generic set schema

Factory Methods

make_bound(manager, schema)

Static factory. Creates a bound layout and allocates fields in the given `FieldManager`.

Parameters: | Parameter | Type | Description | |-----|----|-----| | manager | `FieldManager` | Target field manager | | schema | `*Schema` | Schema instance |

Returns: `BoundLayout*` (concrete subclass)

Example:

```
1 import phynexis
2 pm = phynexis.parsim
3 F = phynexis.fields
4
5 fm = F.FieldManager()
6 schema = pm.NodeKinematicsSchema()
7 layout = pm.BoundLayoutNodeKinematicsSchema.make_bound(fm, schema)
8 print(layout.is_valid()) # True
```

python

Instance Methods

Method	Description
<code>bind(manager)</code>	(Re-)bind to a different field manager
<code>refresh()</code>	Refresh field handles from manager
<code>is_valid()</code>	Returns <code>True</code> if all slots are bound
<code>manager()</code>	Returns the bound <code>FieldManager</code>
<code>handles()</code>	Returns a list of <code>FieldHandle</code> objects
<code>fields()</code>	Returns a list of <code>FieldBase*</code> objects
<code>slot_fields(slot)</code>	Returns fields for a named slot
<code>slot_fields_count(slot)</code>	Returns number of fields in a slot
<code>slot_fields_offset(slot)</code>	Returns offset of a slot in the handle list

Example:

```
1 # Access fields via the layout
```

python

```
2 fields = layout.fields()
3 print(len(fields))
4
5 # Access a specific slot
6 pos_fields = layout.slot_fields("pos")
7 print(len(pos_fields))
```

Known Issues

`fields()` returns `FieldBase*` without RTTI resolution

`layout.fields()` returns a list of `FieldBase*` pointers. The concrete subclass (e.g., `ScalarField`) is not resolved via RTTI in the current binding.

Workaround: Use `layout.handles()` to get `FieldHandle` objects, then retrieve fields via `manager.get_field(handle)`.

Status: noted

Simulator

Python: `phynexis.parsim` **Headers:** `src/parsim/simulator/*.hpp`, `src/parsim/context/*.hpp`

Simulation driver, context management, and operator system.

Simulator

Main simulation driver.

Constructors

Signature	Description
<code>Simulator()</code>	Default simulator.

Methods

Method	Returns	Description
<code>step()</code>	—	Advance one time step
<code>run()</code>	—	Run until end condition
<code>context()</code>	<code>Context</code>	Simulation context
<code>operators()</code>	<code>OperatorSystem</code>	Operator system
<code>config()</code>	<code>SimulationConfig</code>	Current configuration
<code>save_to(path, file, opt)</code>	—	Save state
<code>load_from(path, file, opt)</code>	—	Load state

Example:

```
1 import phynexis
```

python

```
2 pm = phynexis.parsim
3
4 sim = pm.Simulator()
5 ctx = sim.context()
6 print(type(ctx).__name__) # Context
```

SimulationConfig

Configuration struct.

Attribute	Type	Description
time_step	float	Simulation time step
end_time	float	Simulation end time
max_steps	int	Maximum number of steps
enable_gravity	bool	Enable gravity
gravity	Vec3d	Gravity vector
enable_logging	bool	Enable logging

Context

Central simulation context managing all resources.

Methods

Method	Returns	Description
initialize()	—	Initialize context
finalize()	—	Finalize context
graph()	ComputationalGraph	The computational graph
domain()	DomainSystem	Domain decomposition system
state()	RuntimeState	Runtime state
spatial_index()	SpatialIndex	Spatial index (may be None)
ensure_spatial_index()	—	Ensure spatial index is created
find_resource(name)	tuple	Find resource handle (index, epoch)
resource(name)	object	Get typed resource (4 hardcoded keys)
save_to(path, file, opt)	—	Save context
load_from(path, file, opt)	—	Load context

Resource keys: - "computational_graph" → ComputationalGraph - "domain_system" → DomainSystem - "runtime_state" → RuntimeState - "uniform_grid_index" → UniformGridIndex

Example:

```
1 ctx = pm.Context()
2 ctx.initialize()
3 g = ctx.graph()
4 print(g.is_empty()) # True
```

python

OperatorSystem

Container for simulation operators.

Methods

Method	Description
<code>insert(op)</code>	Insert an operator
<code>insert_with_label(label, op)</code>	Insert with a label
<code>get(label)</code>	Get operator by label
<code>get(handle)</code>	Get operator by handle
<code>erase(label)</code>	Remove operator by label
<code>enable(handle)</code>	Enable operator
<code>disable(handle)</code>	Disable operator
<code>size()</code>	Number of operators
<code>empty()</code>	True if no operators
<code>clear()</code>	Remove all operators

Known issue: `BaseOperator` is not bound in Python, so `insert()` is only usable with C++-created operators. The `operators` submodule exposes concrete operators but their insertion API is incomplete.

Status: noted

OperatorSystemEntry

Metadata for an operator.

Attribute	Type	Description
<code>op</code>	<code>handle</code>	Operator handle
<code>label</code>	<code>str</code>	Operator label

RuntimeState

Time and step tracking.

Methods

Method	Returns	Description
<code>step()</code>	<code>int</code>	Current step
<code>time()</code>	<code>float</code>	Current time
<code>time_step()</code>	<code>float</code>	Time step size
<code>set_step(v)</code>	—	Set step
<code>set_time(v)</code>	—	Set time
<code>set_time_step(v)</code>	—	Set time step
<code>advance(dt)</code>	—	Advance time by dt
<code>history()</code>	—	Get history recorder

DomainSystem

Spatial domain decomposition for MPI.

Methods

Method	Returns	Description
<code>initialize()</code>	—	Initialize domain decomposition
<code>config()</code>	<code>DomainSystemConfig</code>	Configuration
<code>subdomain_count()</code>	<code>int</code>	Number of subdomains
<code>domain_bound()</code>	<code>BoundingBox</code>	Global domain bounds
<code>self_subdomain_id()</code>	<code>int</code>	Local subdomain ID
<code>self_subdomain_bound()</code>	<code>BoundingBox</code>	Local subdomain bounds
<code>get_overlapped_subdomain(position)</code>	<code>int</code>	Subdomain overlapping position
<code>is_judge_domain(bbox_a, bbox_b)</code>	<code>bool</code>	Whether this rank judges bbox pair

DomainSystemConfig

Attribute	Type	Description
<code>min_bound</code>	<code>Vec3d</code>	Domain minimum corner
<code>max_bound</code>	<code>Vec3d</code>	Domain maximum corner
<code>subdivisions</code>	<code>Vec3i</code>	Subdivision counts per axis

SpatialIndex

Abstract base for spatial indexing.

Methods

Method	Returns	Description
<code>build(nodes)</code>	—	Build index from <code>NodeSet</code>
<code>query(position, radius)</code>	<code>list[int]</code>	Query nodes within radius
<code>update()</code>	—	Update index

UniformGridIndex

Uniform grid spatial index.

Methods

Method	Returns	Description
<code>build(nodes)</code>	—	Build index from <code>NodeSet</code>
<code>query(position, radius)</code>	<code>list[int]</code>	Query nodes within radius
<code>update()</code>	—	Update index
<code>config()</code>	<code>UniformGridIndexConfig</code>	Current config

UniformGridIndexConfig

Attribute	Type	Description
min_bound	Vec3d	Grid minimum corner
max_bound	Vec3d	Grid maximum corner
grid_size	Vec3d	Cell size
update_mode	str	Update strategy

ResourceRegistry / ContextResource

Resource factory pattern for extensible context resources.

ResourceRegistry

Method	Description
<code>register_factory(name, factory)</code>	Register a resource factory
<code>create(name)</code>	Create a resource by name

ContextResource

Base class for context-managed resources.

Known Issues

`OperatorSystem.insert()` unusable from Python

`BaseOperator` is not exposed in Python bindings, so operators created in Python cannot be inserted into `OperatorSystem`.

Workaround: Use C++-side operator creation or the `Simulator` factory methods.

Status: noted

`Context.resource()` only supports 4 hardcoded keys

Extensible resources fall through to an exception.

Status: noted

phynexis.utils

Utility module providing helper classes and functions for console logging, string manipulation, time measurement, signed distance fields, mesh I/O, sampling, math, and more.

Import

```
1 import phynexis
2
3 # Access submodule directly
4 phynexis.utils.info("hello")
5
```

python

```

6 # Or bind locally
7 from phynexis import utils
8 utils.info("hello")

```

Module Overview

Class / Function	Description
<code>Config</code>	Console output configuration
<code>debug / info / warning / error / fatal / plain</code>	Logging functions
<code>set_level / set_verbose / set_color</code>	Console control functions
<code>to_string</code>	Numeric to string conversion
<code>get_time_micros</code>	High-resolution timestamp
<code>MiniMapIdx64</code>	Lightweight index map
<code>SDFCalculator</code>	Signed distance field calculator
<code>LevelSetField</code>	Level set field representation
<code>STLModel</code>	3D triangle mesh from STL/OFF
<code>Voronoi</code>	Voronoi tessellation (Tessellation / Diagram)
<code>Cork</code>	Boolean mesh operations (CorkWrapper)
<code>OpenMP</code>	Thread control bindings
<code>Vec2d / Vec3d / Vec3i / Vec3u / Vec4d / Vec6d</code>	Fixed-size vectors
<code>Mat2d / Mat3d</code>	Fixed-size matrices
<code>FlatHashMap</code>	Flat hash map
<code>Shape</code>	Geometric shapes (Sphere, Cuboid, Ellipsoid, Cylinder, ...)
<code>SurfaceMesh</code>	Surface mesh struct
<code>DataIO</code>	Tabular data I/O
<code>SaveOptions / LoadOptions</code>	File save/load options
<code>math</code>	Math constants, vector ops, quaternion
<code>sampling</code>	Distribution, spatial, orientation, spherical sampling
<code>vtk</code>	VTK file I/O
<code>wrappers</code>	Cork, CGAL, Eigen, igl, TetGen wrappers

C++ Namespace

`phynexis::utils`

pybind Module

`pyutils` (lazy-loaded via `phynexis.utils`)

See also

- [Manual home](#) — installation, basic usage, CFD–DEM, visualization
- [Python API overview](#) — submodule map and coverage notes

- [phynexis.fields](#) — field types and layouts
- [Deprecated flat references](#) — legacy v0 pages

Console

C++: `phynexis::utils::console` **Python:** `phynexis.utils` **Header:** `src/utils/console.hpp`

Console logging and output control for phynexis. Supports leveled output (debug/info/warning/error/fatal), colored terminal output, timestamps, and MPI rank display.

Config

C++: `phynexis::utils::console::Config` **Python:** `phynexis.utils.Config`

Configuration object controlling the appearance and filtering of console output.

Constructor

`Config()`

Creates a config with default settings (min_level=info, timestamps on, colors on).

Properties

Property	Type	Access	Description
<code>min_level</code>	<code>int</code>	read/write	Minimum log level to display
<code>show_timestamp</code>	<code>bool</code>	read/write	Show timestamps in output
<code>show_level</code>	<code>bool</code>	read/write	Show level tag (INFO, WARN, etc.)
<code>show_location</code>	<code>bool</code>	read/write	Show file/line location
<code>color_output</code>	<code>bool</code>	read/write	Enable ANSI color codes
<code>show_rank</code>	<code>bool</code>	read/write	Show MPI rank prefix

Methods

`set_min_level(level)`

Set minimum display level. Levels: 0=debug, 1=info, 2=warning, 3=error, 4=fatal.

Parameters: | Parameter | Type | Description | |---|---|---| | `level` | `int` | Minimum level to display |

`set_show_timestamp(enabled)`

Toggle timestamp display.

`set_show_level(enabled)`

Toggle level tag display.

`set_show_location(enabled)`

Toggle source location display.

`set_color_output(enabled)`

Toggle colored output.

Example:

```

1 import phynexis
2
3 cfg = phynexis.utils.Config()
4 cfg.set_show_timestamp(False)
5 cfg.set_color_output(False)
6 phynexis.utils.custom(cfg)
7
8 phynexis.utils.info("no timestamp, no color")

```

python

Output:

```

1 [INFO] no timestamp, no color

```

text

Logging Functions

Module-level functions for leveled console output.

debug(msg)

Print a debug-level message. Only visible when `min_level ≤ 0`.

info(msg)

Print an info-level message.

plain(msg)

Print a plain message without any level prefix or formatting.

warning(msg)

Print a warning-level message.

error(msg)

Print an error-level message.

fatal(msg)

Print a fatal-level message and terminate.

Parameters: | Parameter | Type | Description | |—|—|—| | `msg` | `str` | Message string to print |

Broadcast Variants

Each level has an `_all` variant that broadcasts to all MPI ranks:

- `debug_all(msg)`
- `info_all(msg)`
- `warning_all(msg)`
- `error_all(msg)`
- `fatal_all(msg)`

Example:

```

1 import phynexis
2
3 phynexis.utils.info("simulation started")
4 phynexis.utils.debug("debug info: dt=1e-5")
5 phynexis.utils.warning("mesh quality low")

```

python

```
6 phynexis.utils.plain("raw output without prefix")
```

Output:

```
1 [INFO] [12:01:05.232] simulation started
2 [DEBUG] [12:01:05.233] debug info: dt=1e-5
3 [WARNING] [12:01:05.233] mesh quality low
4 raw output without prefix
```

text

Control Functions

set_level(level)

Set the global minimum log level.

set_verbose(enabled)

Enable or disable verbose (debug) output. Equivalent to `set_level(0)`.

set_color(enabled)

Enable or disable colored output globally.

custom(config)

Apply a custom `Config` object to the console.

Example:

```
1 import phynexis
2
3 phynexis.utils.set_level(2)  # Only warning and above
4 phynexis.utils.info("hidden")
5 phynexis.utils.warning("visible")
```

python

Output:

```
1 [WARNING] visible
```

text

Cork

C++: `phynexis::utils::wrappers::CorkWrapper` **Python:** `phynexis.utils.wrappers.CorkWrapper` **Header:**
`src/utils/wrappers/cork_wrapper.hpp`

Boolean operations on triangle meshes (intersection, union, difference, XOR) via the Cork library.

Description

`CorkWrapper` has no default constructor in Python. It is typically accessed through the `wrappers` submodule or used via the `Tessellation` class when `use_cork=True`.

Methods

`mesh_intersect(v1, f1, v2, f2, out_v, out_f, out_info)`

Compute intersection of two meshes.

`mesh_union(v1, f1, v2, f2, out_v, out_f, out_info)`

Compute union of two meshes.

`mesh_difference(v1, f1, v2, f2, out_v, out_f, out_info)`

Compute difference (mesh1 – mesh2).

`mesh_xor(v1, f1, v2, f2, out_v, out_f, out_info)`

Compute symmetric difference.

Parameters: | Parameter | Type | Description | |---|---|---| | `v1, v2` | `list[Vec3d]` | Input mesh vertices | | `f1, f2` | `list[Vec3i]` | Input mesh faces | | `out_v` | `list[Vec3d]` | Output vertices (inout) | | `out_f` | `list[Vec3i]` | Output faces (inout) | | `out_info` | `list[int]` | Output labels (inout, optional) |

Note

- The overload without `out_info` is also available.
 - A specialized `mesh_intersect` with tolerance and direction exists for ray-like intersection tests.
 - `CorkWrapper` instance access in Python is currently limited. Consider using STLModel-based boolean operations if available.
-

Unexposed C++ API

- Default constructor for direct instantiation

DataIO

C++: `phynexis::utils::DataIO` **Python:** `phynexis.utils.DataIO` **Header:** `src/utils/serde/data_io.hpp`

Utility class for importing and exporting tabular data (CSV, TSV, space-separated) with automatic delimiter detection.

Methods

`import_data(filename, lines_to_skip=0)`

Import numeric data from a file. Supports comma, tab, and space delimiters with auto-detection.

Parameters: | Parameter | Type | Default | Description | |-----|---|-----|-----| | `filename` | `str` | — | Path to data file | | `lines_to_skip` | `int` | 0 | Number of header lines to skip |

Returns: `list[list[float]]` — 2D array of doubles

save_data(data, head, filename)

Save tabular data to a file with the given header string.

Parameters: | Parameter | Type | Description | |-----|----|-----| | data | list[list[int]] or list[list[float]] | 2D data array | | head | str | Header line (e.g. "x,y,z") | | filename | str | Output file path |

file_exist(filename)

Check if a file exists.

Parameters: | Parameter | Type | Description | |-----|----|-----| | filename | str | File path to check |

Returns: bool

Example

```
1 import phynexis
2
3 # Save integer data
4 phynexis.utils.DataIO.save_data(
5     [[1, 2, 3], [4, 5, 6]],
6     "x,y,z",
7     "/tmp/data_int.csv"
8 )
9
10 # Save float data
11 phynexis.utils.DataIO.save_data(
12     [[1.5, 2.5, 3.5], [4.5, 5.5, 6.5]],
13     "x,y,z",
14     "/tmp/data_float.csv"
15 )
16
17 # Import data (auto-detects delimiter)
18 data = phynexis.utils.DataIO.import_data("/tmp/data_float.csv")
19 print("rows:", len(data))
20 print("first row:", data[0])
21 print("second row:", data[1])
22
23 # File existence check
24 print("exists:", phynexis.utils.DataIO.file_exist("/tmp/data_float.csv"))
25 print("missing:", phynexis.utils.DataIO.file_exist("/tmp/nonexistent.csv"))
```

Output:

```
1 rows: 2
2 first row: [1.5, 2.5, 3.5]
3 second row: [4.5, 5.5, 6.5]
4 exists: True
5 missing: False
```

Unexposed C++ API

- detect_delimiter() / parse_line() — internal helpers

FlatHashMap

C++: `phynexis::utils::FlatHashMap<Key, Value>` **Python:** `phynexis.utils.FlatHashMap` (alias for `FlatHashMap64_32`) **Header:** `src/utils/types/flat_hash_map.hpp`

Flat hash map with open addressing. Optimized for cache-friendly iteration and small data performance. Exposed as template instantiations with Pythonic dict-like interface.

Available Instantiations

Python Name	Key Type	Value Type	Description
<code>FlatHashMap</code>	<code>int64</code>	<code>int32</code>	Default alias (64→32)
<code>FlatHashMap64_32</code>	<code>int64</code>	<code>int32</code>	64-bit key, 32-bit value
<code>FlatHashMap64_64</code>	<code>int64</code>	<code>int64</code>	64-bit key, 64-bit value
<code>FlatHashMap32_32</code>	<code>int32</code>	<code>int32</code>	32-bit key, 32-bit value
<code>FlatHashMap64_64i</code>	<code>int64</code>	<code>int64</code>	Alias for <code>FlatHashMap64_64</code>

Constructor

`FlatHashMap()`

Creates an empty hash map with default capacity.

`FlatHashMap(bucket_count)`

Creates a hash map with pre-allocated bucket count.

Parameters: | Parameter | Type | Description | |---|---|---| | `bucket_count` | `int` | Initial number of buckets |

Methods

`empty()`

Return True if the map contains no elements.

`size() / __len__()`

Return the number of key-value pairs.

`max_size()`

Return the maximum possible size.

`reserve(count)`

Pre-allocate space for at least `count` elements.

`clear()`

Remove all elements.

insert(key, value)

Insert a key-value pair. Returns (inserted, value) tuple.

erase(key)

Remove the element with the given key. Returns number of elements removed (0 or 1).

find(key)

Lookup a key. Returns (found, value) tuple.

contains(key) / key in map

Check if a key exists.

count(key)

Return 1 if key exists, 0 otherwise.

at(key)

Access value by key with bounds checking.

load_factor()

Return current load factor (size / capacity).

max_load_factor() / max_load_factor(ml)

Get or set the maximum load factor threshold for rehashing.

rehash(count)Force rehash to at least `count` buckets.

Python Protocols

map[key] (getitem)

Access or default-insert a value.

map[key] = value (setitem)

Insert or update a key-value pair.

key in map (contains)

Check membership.

for k, v in map (iteration)

Iterate over key-value pairs.

len(map)

Return element count.

Example

```

1 import phynexis
2
3 # Create default map (int64 -> int32)

```

python

```

4 m = phynexis.utils.FlatHashMap()
5 print("empty:", m.empty())
6
7 # Insert via setitem
8 m[10] = 100
9 m[20] = 200
10 m[30] = 300
11
12 print("size:", m.size())
13 print("m[10]:", m[10])
14 print("m.at(20):", m.at(20))
15 print("contains 10:", m.contains(10))
16 print("contains 99:", 99 in m)
17 print("count 10:", m.count(10))
18 print("len:", len(m))
19
20 # find
21 found, val = m.find(10)
22 print("find(10):", found, val)
23 found2, val2 = m.find(99)
24 print("find(99):", found2, val2)
25
26 # insert method
27 result = m.insert(40, 400)
28 print("insert(40, 400):", result)
29
30 # items / keys / values
31 print("items:", m.items())
32 print("keys:", m.keys())
33 print("values:", m.values())
34
35 # iteration
36 for k, v in m:
37     print(f" {k} -> {v}")
38
39 # erase
40 m.erase(10)
41 print("after erase 10, size:", m.size())
42
43 # load factor
44 print("load_factor:", m.load_factor())
45
46 # reserve and rehash
47 m.reserve(100)
48 print("after reserve, size:", m.size())
49 m.rehash(256)
50 print("after rehash, load_factor:", m.load_factor())
51
52 # clear
53 m.clear()
54 print("after clear, size:", m.size())
55
56 # Bucket count constructor
57 m2 = phynexis.utils.FlatHashMap(64)
58 print("m2 size:", m2.size())
59
60 # 64->64 variant
61 m64 = phynexis.utils.FlatHashMap64_64()
62 m64[1] = 100

```

```

63 m64[2] = 200
64 print("64_64 size:", m64.size())
65
66 # 32->32 variant
67 m32 = phynexis.utils.FlatHashMap32_32()
68 m32[1] = 100
69 print("32_32 size:", m32.size())

```

Output:

```

1 empty: True
2 size: 3
3 m[10]: 100
4 m.at(20): 200
5 contains 10: True
6 contains 99: False
7 count 10: 1
8 len: 3
9 find(10): True 100
10 find(99): False 0
11 insert(40, 400): (True, 400)
12 items: [(20, 200), (10, 100), (30, 300), (40, 400)]
13 keys: [20, 10, 30, 40]
14 values: [200, 100, 300, 400]
15 20 -> 200
16 10 -> 100
17 30 -> 300
18 40 -> 400
19 after erase 10, size: 3
20 load_factor: 0.046875
21 after reserve, size: 3
22 after rehash, load_factor: 0.01171875
23 after clear, size: 0
24 m2 size: 0
25 64_64 size: 2
26 32_32 size: 1

```

text

Unexposed C++ API

The following methods are not yet exposed to Python:

- Copy/move constructors and assignment operators
- `emplace()` / `try_emplace()` — template methods difficult to bind
- Iterator-based `erase(pos)` — iterator not fully exposed
- `cbegin()` / `cend()` — const iterators

Binding Fix Record

2026-05-03: `items()` / `keys()` / `values()` originally returned `VecX<T>` which threw `TypeError` because the type was not registered. Changed to return `py::list`. Issue fixed. See `discovery-log.md` for details.

LevelSetField

C++: phynexis::utils::LevelSetField **Python:** phynexis.utils.LevelSetField **Header:** src/utils/level_set_field.hpp

Represents a level set function ϕ on a regular 3D grid. Used for implicit surface representation, interface tracking, and shape operations.

Constructor

LevelSetField()

Creates a level set field with default grid parameters: - Corner: $(-0.5, -0.5, -0.5)$ - Spacing: 0.05 - Dimensions: $(21, 21, 21)$

Methods

get_corner()

Return the grid corner position.

Returns: Vec3d

get_spacing()

Return the grid spacing.

Returns: float

get_dimensions()

Return the grid dimensions.

Returns: Vec3i

set_corner(corner)

Set the grid corner.

Parameters: | Parameter | Type | Description | |---|---|---| | `corner` | Vec3d | Lower corner of the grid |

set_spacing(sp)

Set the grid spacing.

Parameters: | Parameter | Type | Description | |---|---|---| | `sp` | float | Cell spacing |

set_dimension(dim)

Set the grid dimensions.

Parameters: | Parameter | Type | Description | |---|---|---| | `dim` | Vec3u | Number of cells in each dimension |

get_phi(id) / set_phi(id, value) / phi(i, j, k)

Access level set values at grid cells.

Parameters: | Parameter | Type | Description | |——|——|——| | id | Vec3u | Grid cell index | | i, j, k | int | Individual indices | | value | float | New phi value |

`init_from_sdf_calculator(sdf)`

Initialize phi values by sampling an SDFCalculator.

`signed_distance(pos)`

Interpolate signed distance at an arbitrary point in space.

`gradient_interpolate(pos)`

Interpolate gradient at a point.

`gradient_minus(id) / gradient_plus(id)`

Compute gradient at a grid point using backward/forward differences.

`reinitialization() / reinitialization(iterations, dt)`

Reinitialize the level set function to maintain signed distance property.

`is_initialized()`

Check if phi_table has been allocated.

Returns: bool

Example

```
1 import phynexis
2
3 # Create and configure grid
4 lsf = phynexis.utils.LevelSetField()
5 lsf.set_corner(phynexis.utils.Vec3d(0.0, 0.0, 0.0))
6 lsf.set_spacing(0.1)
7 lsf.set_dimension(phynexis.utils.Vec3u(11, 11, 11))
8
9 print("corner:", lsf.get_corner())
10 print("spacing:", lsf.get_spacing())
11 print("dimensions:", lsf.get_dimensions())
12
13 # Set phi values
14 lsf.set_phi(phynexis.utils.Vec3u(5, 5, 5), 1.0)
15 print("phi at center:", lsf.phi(5, 5, 5))
16
17 # Query signed distance
18 pos = phynexis.utils.Vec3d(0.5, 0.5, 0.5)
19 print("sd at", pos, ":", lsf.signed_distance(pos))
```

python

Output:

```
1 corner: Vec3d(0, 0, 0)
2 spacing: 0.1
3 dimensions: Vec3i(11, 11, 11)
4 phi at center: 1.0
5 sd at Vec3d(0.5, 0.5, 0.5) : -0.5
```

text

Unexposed C++ API

- `get_phi_array()` — FlatArray3D wrapper
- `get_phi_table()` — Direct vector access
- `index_3d(i, j, k)` — Flat index computation

math

C++: `phynexis::utils::math` **Python:** `phynexis.utils.math` **Header:** `src/utils/math/math.hpp`

Mathematical constants, vector operations, and quaternion utilities.

Constants

Name	Value	Description
PI	3.14159...	Circle constant
EPSILON	2.22e-16	Machine epsilon for doubles
TOL	1e-6	Default numerical tolerance
HUGE_VALUE	1e20	Large sentinel value
SQRT_2	1.41421...	Square root of 2
SQRT_3	1.73205...	Square root of 3

Vector Operations

`dot(a, b)`

Compute dot product of two vectors.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `a` | `Vec3d` | First vector | | `b` | `Vec3d` | Second vector |

Returns: `float`

`cross(a, b)`

Compute cross product of two vectors.

Returns: `Vec3d`

`norm_l2(v)`

Compute L2 norm (magnitude) of a vector.

Returns: `float`

`normalize(v)`

Normalize a vector in-place.

Quaternion

Module `phynexis.utils.math.quaternion` provides quaternion operations.

`from_rodrigues(angle, axis)`

Create a quaternion from angle-axis (Rodrigues) representation.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `angle` | `float` | Rotation angle in radians | | `axis` | `Vec3d` | Rotation axis |

Returns: `Vec4d`

`to_rodrigues(q) / to_matrix(q) / from_matrix(m)`

Convert quaternion to/from Rodrigues, rotation matrix.

`multiply(a, b) / conjugate(q) / normalize(q) / add(a, b)`

Quaternion arithmetic.

Example

```
1 import phynexis
2
3 # Constants
4 print("PI:", phynexis.utils.math.PI)
5 print("EPSILON:", phynexis.utils.math.EPSILON)
6
7 # Vector ops
8 a = phynexis.utils.Vec3d(1.0, 0.0, 0.0)
9 b = phynexis.utils.Vec3d(0.0, 1.0, 0.0)
10 print("dot:", phynexis.utils.math.dot(a, b))
11 print("cross:", phynexis.utils.math.cross(a, b))
12 print("norm:", phynexis.utils.math.norm_l2(a))
13
14 # Quaternion
15 q = phynexis.utils.math.quaternion.from_rodrigues(
16     phynexis.utils.math.PI / 2,
17     phynexis.utils.Vec3d(0.0, 0.0, 1.0)
18 )
19 print("quaternion:", q)
```

python

Output:

```
1 PI: 3.141592653589793
2 EPSILON: 2.220446049250313e-16
3 dot: 0.0
4 cross: Vec3d(0, 0, 1)
5 norm: 1.0
6 quaternion: Vec4d(0, 0, 0.707107, 0.707107)
```

text

Unexposed C++ API

- `legendre_p`, `calculate_ynm_fast`, `normalization_factor` — spherical harmonics functions
- `flat_array3d` utilities

MiniMap

C++: `phynexis::utils::MiniMap<Int64, Int64>` **Python:** `phynexis.utils.MiniMapIdx64` **Header:** `src/utils/mini_map.hpp`

Lightweight hash map optimized for index-to-index lookups. Uses flat hash map internally for cache-friendly access.

Constructor

MiniMapIdx64()

Creates an empty map.

Methods

exist(key)

Check if a key exists in the map.

Parameters: | Parameter | Type | Description | |---|---|---| | `key` | `int` | Key to look up |

Returns: `bool` — True if key exists

size()

Return the number of entries.

Returns: `int` — Entry count

clear()

Remove all entries.

Python Protocols

map[key] (getitem)

Retrieve value by key. Raises `KeyError` if key not found.

map[key] = value (setitem)

Insert or update a key-value pair.

key in map (contains)

Check membership. Equivalent to `exist(key)`.

Example

```
1 import phynexis
2
3 m = phynexis.utils.MiniMapIdx64()
4
5 # Insert entries
```

python

```

6  m[10] = 100
7  m[20] = 200
8  m[30] = 300
9
10 print(f"size: {m.size()}")
11 print(f"m[10] = {m[10]}")
12 print(f"20 in map: {20 in m}")
13 print(f"99 in map: {99 in m}")
14 print(f"exist(10): {m.exist(10)}")
15
16 # Update
17 m[10] = 999
18 print(f"updated m[10] = {m[10]}")
19
20 # Clear
21 m.clear()
22 print(f"after clear: {m.size()}")

```

Output:

```

1  size: 3
2  m[10] = 100
3  20 in map: True
4  99 in map: False
5  exist(10): True
6  updated m[10] = 999
7  after clear: 0

```

text

Unexposed C++ API

MiniMap is a template class. Only the `Int64→Int64` instantiation (`MiniMapIdx64`) is exposed. Other instantiations (e.g., `int→double`) are not available in Python.

OpenMP

C++: `phynexis::utils::parallel` / OpenMP runtime **Python:** `phynexis.utils` **Header:** `src/utils/parallel/parallel.hpp`

Thread-level parallelism control. These functions map directly to OpenMP runtime calls.

Functions

`omp_get_max_threads()`

Return the maximum number of threads available.

Returns: `int`

`omp_set_num_threads(num_threads)`

Set the number of threads for subsequent parallel regions.

Parameters: | Parameter | Type | Description | |---|---|---| | `num_threads` | `int` | Number of threads |

`parallel_max_threads()`

Alias for `omp_get_max_threads()`.

`parallel_set_num_threads(num_threads)`

Alias for `omp_set_num_threads()`.

Example

```
1 import phynexis
2
3 print("max threads:", phynexis.utils.omp_get_max_threads())
4 phynexis.utils.omp_set_num_threads(4)
5 print("set to 4 threads")
```

python

Output:

```
1 max threads: 10
2 set to 4 threads
```

text

Note

- These functions affect the **global** OpenMP thread pool.
- Setting threads via `parallel_set_num_threads()` also calls `omp_set_num_threads()` internally.

sampling

C++: `phynexis::utils::sampling` **Python:** `phynexis.utils.sampling` **Header:** `src/utils/sampling/sampling.hpp`

Sampling utilities for distributions, spatial points, orientations, and spherical arrangements.

Distribution Sampling

Class	Description
<code>UniformDistribution(min, max)</code>	Uniform distribution in <code>[min, max]</code>
<code>Normal(mean, std)</code>	Normal (Gaussian) distribution
<code>Exponential(lambda)</code>	Exponential distribution

Spatial Sampling

Spatial samplers generate points within a shape's bounding volume.

Class	Description
<code>Random(shape)</code>	Random uniform sampling inside a shape
<code>Grid(shape)</code>	Regular grid sampling
<code>CVT(shape)</code>	Centroidal Voronoi tessellation sampling
<code>WCVT(shape)</code>	Weighted CVT sampling
<code>Voronoi(shape)</code>	Voronoi-based sampling

Orientation Sampling

Class	Description
<code>Uniform</code>	Uniform random orientations
<code>ConeDir</code>	Orientations within a cone
<code>ConeRef</code>	Orientations relative to a reference direction

Spherical Sampling

Class	Description
<code>Uniform</code>	Uniform points on sphere
<code>CVT</code>	Spherical CVT
<code>WCVT</code>	Weighted spherical CVT
<code>GoldenSpiral</code>	Fibonacci/golden spiral point distribution

Example

```

1 import phynexis
2
3 # Distribution sampling
4 u = phynexis.utils.sampling.UniformDistribution(0.0, 1.0)
5 n = phynexis.utils.sampling.Normal(0.0, 1.0)
6 e = phynexis.utils.sampling.Exponential(1.0)
7 print("Uniform:", u)
8 print("Normal:", n)
9 print("Exponential:", e)
```

python

Output:

```

1 Uniform: <phynexis.lib.pyutils.sampling.distribution.Uniform object at 0x...>
2 Normal: <phynexis.lib.pyutils.sampling.distribution.Normal object at 0x...>
3 Exponential: <phynexis.lib.pyutils.sampling.distribution.Exponential object at 0x...>
```

text

Unexposed C++ API

- Distribution `sample()` methods returning scalar values
- Spatial samplers' `get_samples()` methods returning `VecX<Vec3d>` (unregistered)
- Orientation and spherical sampler detail methods
- Factory for creating samplers by name

SaveOptions / LoadOptions

C++: `phynexis::utils::SaveOptions, LoadOptions, SaveVtkOptions, FileFormat` **Python:**

`phynexis.utils.SaveOptions, LoadOptions, SaveVtkOptions, FileFormat` **Header:** `src/utils/serde/save_options.hpp`

Configuration objects for file save/load operations used by `Shape.save_to()` / `Shape.load_from()` and other serializable types.

FileFormat

Enum defining supported file formats.

Value	Description
<code>FileFormat.Auto</code>	Infer from file extension
<code>FileFormat.Json</code>	JSON format
<code>FileFormat.Binary</code>	Binary format
<code>FileFormat.VTK</code>	VTK format
<code>FileFormat.Stl</code>	STL mesh format
<code>FileFormat.Off</code>	OFF mesh format

SaveOptions

Constructor

SaveOptions()

Default: `overwrite=True, create_directories=True, format_hint=Auto`.

Properties

Property	Type	Access	Description
<code>overwrite</code>	<code>bool</code>	read/write	Overwrite existing files
<code>create_directories</code>	<code>bool</code>	read/write	Create parent directories
<code>format_hint</code>	<code>FormatHint</code>	read/write	Format override hint

SaveVtkOptions

Inherits from `SaveOptions`. Adds VTK-specific settings.

Constructor

SaveVtkOptions()

Default: `binary=False, use_double=False`.

Properties

Property	Type	Access	Description
binary	bool	read/write	Write VTK in binary mode
use_double	bool	read/write	Use double precision

LoadOptions

Constructor

LoadOptions()

Default: create_directories=False, format_hint=Auto.

Properties

Property	Type	Access	Description
create_directories	bool	read/write	Create parent directories
format_hint	FormatHint	read/write	Format override hint

Example

```

1 import phynexis
2
3 # Save options with JSON format
4 opt = phynexis.utils.SaveOptions()
5 opt.overwrite = True
6 opt.create_directories = True
7 opt.format_hint.value = phynexis.utils.FileFormat.Json
8 print("format:", opt.format_hint.value)
9
10 # VTK options
11 vtk_opt = phynexis.utils.SaveVtkOptions()
12 vtk_opt.binary = True
13 vtk_opt.use_double = True
14 print("vtk binary:", vtk_opt.binary)
15
16 # Load options
17 load_opt = phynexis.utils.LoadOptions()
18 load_opt.create_directories = False
19 print("load create_dirs:", load_opt.create_directories)
```

python

Output:

```

1 format: FileFormat.Json
2 vtk binary: True
3 load create_dirs: False
```

text

SDFCalculator

C++: `phynexis::utils::SDFCalculator` **Python:** `phynexis.utils.SDFCalculator` **Header:** `src/utils/sdf_calculator.hpp`

Computes signed distance fields (SDFs) from triangle meshes using an AABB tree accelerated by libigl. Supports point queries, surface projections, closest facet lookups, and batch evaluation.

Constructor

`SDFCalculator()`

Creates an empty calculator. Must be initialized via `init_from_stl()` before use.

Methods

`init_from_stl(vertices, facets)`

Initialize from raw vertex and face arrays.

Parameters: | Parameter | Type | Description | |---|---|---| | `vertices` | `list[Vec3d]` | Mesh vertex positions | | `facets` | `list[Vec3u]` | Triangle face indices |

`init_from_stl(stl_model)`

Initialize from an `STLModel` object.

Parameters: | Parameter | Type | Description | |---|---|---| | `stl_model` | `STLModel` | Loaded STL model |

`initialize()`

Reset to empty state.

`signed_distance(pos)`

Compute signed distance at a point. Negative = inside, positive = outside (depending on mesh orientation).

Parameters: | Parameter | Type | Description | |---|---|---| | `pos` | `Vec3d` | Query point |

Returns: `float` — Signed distance

`surface_projection(pos)`

Find the closest surface point on the mesh.

Parameters: | Parameter | Type | Description | |---|---|---| | `pos` | `Vec3d` | Query point |

Returns: `Vec3d` — Closest surface point

`closest_facet(pos)`

Find the index of the closest triangle facet.

Parameters: | Parameter | Type | Description | |---|---|---| | `pos` | `Vec3d` | Query point |

Returns: `int` — Facet index

signed_distances(points)

Batch compute signed distances for multiple points.

Parameters: | Parameter | Type | Description | |---|---|---| | points | list[Vec3d] | Query points |

Returns: list[float] — Signed distances

Example

```
1 import phynexis
2
3 # Load a mesh
4 model = phynexis.utils.STLModel()
5 model.init_from_stl("/path/to/sphere.stl")
6
7 # Build SDF
8 sdf = phynexis.utils.SDFCalculator()
9 sdf.init_from_stl(model)
10
11 # Query points
12 center = phynexis.utils.Vec3d(0.0, 0.0, 0.0)
13 outside = phynexis.utils.Vec3d(2.0, 0.0, 0.0)
14
15 print("sd at center:", sdf.signed_distance(center))
16 print("sd at (2,0,0):", sdf.signed_distance(outside))
17 print("surface proj:", sdf.surface_projection(outside))
18 print("closest facet:", sdf.closest_facet(outside))
```

python

Output:

```
1 sd at center: -0.49866546590344585
2 sd at (2,0,0): -2.2589289071338654
3 surface proj: Vec3d(-0.21319, 0.411084, -0.188573)
4 closest facet: 0
```

text

Unexposed C++ API

The following constructors and methods are not yet exposed:

- SDFCalculator(const SDFCalculator&) — Copy constructor
- SDFCalculator(SDFCalculator&&) — Move constructor
- Assignment operators

STLModel

C++: phynexis::utils::STLModel **Python:** phynexis.utils.STLModel **Header:** src/utils/mesh/
stl_model.hpp

3D triangle mesh model supporting STL/OFF file I/O, geometric transformations, decimation, convex hull, and mesh analysis.

Constructor

`STLModel()`

Creates an empty model.

`STLModel(vertices, facets)`

Creates a model from vertex and face arrays.

Parameters: | Parameter | Type | Description | |---|---|---| | `vertices` | `list[Vec3d]` | Vertex positions | | `facets` | `list[Vec3i]` | Triangle face indices |

`STLModel(file_path)`

Load from a file path (auto-detects format).

Properties

Property	Type	Description
<code>vertices</code>	<code>list[Vec3d]</code>	Model vertices (read/write)
<code>facets</code>	<code>list[Vec3i]</code>	Triangle face indices (read/write)

Methods

`init_from_stl(file_path) / init_from_off(file_path)`

Load from STL or OFF file.

`save_to(path, file, options)`

Save model to file with format options.

`transpose() / rotate(rotation)`

Apply geometric transformations.

`copy_pose() / copy_pose_dev()`

Copy the model's current pose.

`remove_unreferenced_vertices() / remove_duplicate_vertices()`

Clean up mesh topology.

`reorient_facets()`

Ensure facet normals point outward.

`decimate(target_num_faces)`

Reduce face count via mesh decimation.

`standardize()`

Center at origin and scale to unit size.

`set_size(size)`

Scale to a specific size.

make_convex()

Convert to convex hull.

refine() / **smooth_mesh(iterations)**

Subdivide or smooth the mesh.

merge(other_model)

Merge with another STLModel.

size()

Return characteristic size (diagonal of bounding box).

get_center() / **get_surface_area()** / **volume()** / **inertia()**

Compute geometric properties.

is_convex() / **is_face_outside(facet_index, point)** / **enclose(point)**

Convexity and containment tests.

bound_aabb()

Return axis-aligned bounding box.

print_info()

Print model summary.

Static Methods

CenterOf(vertices, facets) / **SurfaceAreaOf(vertices, facets)** / **VolumeOf(vertices, facets)** /**InertiaOf(vertices, facets)** / **check_convexity_of(vertices, facets)**

Compute properties from raw mesh data without creating an STLModel instance.

Example

```

1  import phynexis
2
3  # Load from file
4  model = phynexis.utils.STLModel()
5  model.init_from_stl("/path/to/sphere.stl")
6
7  print("size:", model.size())
8  model.print_info()
9
10 # Geometric analysis
11 print("center:", model.get_center())
12 print("volume:", model.volume())
13 print("surface area:", model.get_surface_area())
14 print("is convex:", model.is_convex())
15
16 # Transformations
17 model.standardize()
18 print("after standardize, size:", model.size())

```

python

Output:

```
1 size: 0.08174285936076031
2 is convex: True
3 after standardize, size: 1.0
```

text

Example: Create from vertices and facets

```
1 import phynexis
2
3 model = phynexis.utils.STLModel()
4 model.vertices = [
5     phynexis.utils.Vec3d(0.0, 0.0, 0.0),
6     phynexis.utils.Vec3d(1.0, 0.0, 0.0),
7     phynexis.utils.Vec3d(0.0, 1.0, 0.0),
8 ]
9 model.facets = [phynexis.utils.Vec3i(0, 1, 2)]
10
11 print("vertices:", len(model.vertices))
12 print("facets:", len(model.facets))
13 print("center:", model.get_center())
14 print("volume:", model.volume())
15 print("surface area:", model.get_surface_area())
```

python

Output:

```
1 vertices: 3
2 facets: 1
3 center: Vec3d(0.333333, 0.333333, 0)
4 volume: 0.0
5 surface area: 0.5
```

text

Unexposed C++ API

- `get_triangle_strips()` — return type not fully exposed

StringUtils

C++: `phynexis::utils::to_string` **Python:** `phynexis.utils.to_string` **Header:** `src/utils/string_utils.hpp`

Numeric-to-string conversion with consistent formatting.

Functions

`to_string(value)`

Convert a numeric value to a formatted string.

Overloads: | Signature | Description | |——|——| | `to_string(value: int) -> str` | Integer formatting | | `to_string(value: float) -> str` | Scientific notation for doubles |

Parameters: | Parameter | Type | Description | |——|——|——| | `value` | `int` | `float` | Numeric value to convert |

Returns: `str` — Formatted string

Example:

```
1 import phynexis
2
3 print(phynexis.utils.to_string(42))
4 print(phynexis.utils.to_string(3.14))
5 print(phynexis.utils.to_string(1e-6))
6 print(phynexis.utils.to_string(-2.5e3))
```

python

Output:

```
1 42
2 3.140000e+00
3 1.000000e-06
4 -2.500000e+03
```

text

Unexposed C++ API

The following C++ string utilities are not yet exposed to Python:

- `SplitString` / `JoinString` — string split/join
- `Trim` / `ToLower` / `ToUpper` — string transforms
- `FormatString` — formatted string (C++ `fmt` style)

SurfaceMesh

C++: `phynexis::utils::mesh::SurfaceMesh` **Python:** `phynexis.utils.SurfaceMesh` **Header:** `src/utils/mesh/surface_mesh.hpp`

Minimal struct representing a surface mesh as vertices and triangular facets.

Constructor

SurfaceMesh()

Creates an empty mesh.

Properties

Property	Type	Description
<code>vertices</code>	<code>list[Vec3d]</code>	Vertex positions (read/write)
<code>facets</code>	<code>list[Vec3u]</code>	Triangle face indices (read/write)

Example

```

1 import phynexis
2
3 sm = phynexis.utils.SurfaceMesh()
4 sm.vertices = [
5     phynexis.utils.Vec3d(0.0, 0.0, 0.0),
6     phynexis.utils.Vec3d(1.0, 0.0, 0.0),
7     phynexis.utils.Vec3d(0.0, 1.0, 0.0),
8 ]
9 sm.facets = [phynexis.utils.Vec3u(0, 1, 2)]
10
11 print("vertices:", len(sm.vertices))
12 print("facets:", len(sm.facets))
13 print("first vertex:", sm.vertices[0])

```

python

Output:

```

1 vertices: 3
2 facets: 1
3 first vertex: Vec3d(0, 0, 0)

```

text

Unexposed C++ API

- `SurfaceMesh` is a simple POD struct. No additional methods are defined.

TimeUtils

C++: `phynexis::utils::get_time_micros` **Python:** `phynexis.utils.get_time_micros` **Header:** `src/utils/time_utils.hpp`

High-resolution time measurement utilities.

Functions

`get_time_micros()`

Returns the current time in microseconds since an arbitrary epoch. Useful for measuring elapsed time between two calls.

Returns: `int` — Time in microseconds

Example:

```

1 import phynexis
2 import time
3
4 t0 = phynexis.utils.get_time_micros()
5 time.sleep(0.1) # 100 ms
6 t1 = phynexis.utils.get_time_micros()
7
8 elapsed_ms = (t1 - t0) / 1000.0

```

python

```
9 print(f"elapsed: {elapsed_ms:.2f} ms")
```

Output:

```
1 elapsed: 100.12 ms
```

text

Unexposed C++ API

- `Timer` class — Scoped RAIL timer with automatic logging
- `SleepMicros` / `SleepMillis` — Sleep functions

Vectors and Matrices

C++: `phynexis::utils::Vec2, Vec3, Vec4, Vec6, Mat2, Mat3` **Python:** `phynexis.utils.Vec2d, Vec3d, Vec3i, Vec3u, Vec4d, Vec6d, Mat2d, Mat3d` **Header:** `src/utils/types/vec*.hpp, mat*.hpp`

Small fixed-size vector and matrix types used throughout `phynexis`. Exposed with Pythonic protocols (`len()`, indexing, iteration) where supported.

Vector Types

Python Name	C++ Type	Elements	Component Access	Arithmetic
<code>Vec2d</code>	<code>Vec2<double></code>	2 floats	<code>x, y</code>	<code>+, -, *, /</code> with scalar and vector
<code>Vec2i</code>	<code>Vec2<int></code>	2 ints	<code>x, y</code>	<code>+, -</code>
<code>Vec3d</code>	<code>Vec3<double></code>	3 floats	<code>x, y, z</code>	<code>+, -, *, /</code> with scalar and vector
<code>Vec3i</code>	<code>Vec3<int></code>	3 ints	<code>x, y, z</code>	<code>+, -</code>
<code>Vec3u</code>	<code>Vec3<Int64></code>	3 ints	<code>x, y, z</code>	<code>+, -</code>
<code>Vec4d</code>	<code>Vec4<double></code>	4 floats	<code>x, y, z, w</code>	<code>+, -, *, /</code> with scalar and vector
<code>Vec4i</code>	<code>Vec4<int></code>	4 ints	<code>x, y, z, w</code>	<code>+, -</code>
<code>Vec6d</code>	<code>Vec6<double></code>	6 floats	<code>xx, yy, zz, xy, xz, yz</code>	—

Constructors

`Vec2d(x, y)` / `Vec2i(x, y)`

`Vec3d(x, y, z)` / `Vec3i(x, y, z)` / `Vec3u(x, y, z)`

`Vec4d(x, y, z, w)` / `Vec4i(x, y, z, w)`

Note: The C++ `Vec4` constructor order is `(w, x, y, z)`, but the Python binding accepts `(x, y, z, w)` in the `repr` output order. After construction, `v.x, v.y, v.z, v.w` correspond to the expected components.

`Vec6d(xx, yy, zz, xy, xz, yz)`

Symmetric tensor storage (e.g. stress/strain components).

Properties / Indexing

Feature	Vec2d/3d/4d/2i/3i/4i/3u	Vec6d
<code>len(v)</code>	Yes	No
<code>v[i] / v[i] = val</code>	Yes	No
Named components	Yes	Yes
<code>for x in v</code>	Yes (iteration)	No

Methods

`dot(other)` — Vec3d only

Compute dot product.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `other` | Vec3d | Other vector |

Returns: float

`cross(other)` — Vec3d only

Compute cross product.

Parameters: | Parameter | Type | Description | |-----|----|-----| | `other` | Vec3d | Other vector |

Returns: Vec3d

Matrix Types

Python Name	C++ Type	Shape	Row Access
<code>Mat2d</code>	<code>Mat2<double></code>	2x2	<code>mat[i]</code> returns Vec2d
<code>Mat3d</code>	<code>Mat3<double></code>	3x3	<code>mat[i]</code> returns Vec3d

Constructors

`Mat2d()` / `Mat3d()`

Creates a zero-initialized matrix.

Indexing

Feature	Mat2d	Mat3d
<code>mat[i]</code>	Returns Vec2d	Returns Vec3d
<code>mat[i] = row_vec</code>	Yes	Yes

Example

```

1 import phynexis
2
3 # Vec3d — most commonly used
4 v = phynexis.utils.Vec3d(1.0, 2.0, 3.0)
5 print("v:", v)
6 print("len:", len(v))
7 print("x:", v.x, "y:", v.y, "z:", v.z)
8 print("v[0]:", v[0], "v[1]:", v[1], "v[2]:", v[2])

```

python

```

9
10 # Element access and mutation
11 v.x = 10.0
12 v[1] = 20.0
13 print("after mutation:", v)
14
15 # Arithmetic
16 a = phynexis.utils.Vec3d(1.0, 0.0, 0.0)
17 b = phynexis.utils.Vec3d(0.0, 1.0, 0.0)
18 print("a + b:", a + b)
19 print("a * 2.0:", a * 2.0)
20 print("2.0 * a:", 2.0 * a)
21
22 # Dot and cross
23 print("dot:", a.dot(b))
24 print("cross:", a.cross(b))
25
26 # Vec3i
27 vi = phynexis.utils.Vec3i(1, 2, 3)
28 print("Vec3i:", vi, "len:", len(vi))
29
30 # Vec6d - symmetric tensor
31 s = phynexis.utils.Vec6d(1.0, 2.0, 3.0, 0.5, 0.6, 0.7)
32 print("Vec6d xx:", s.xx, "xy:", s.xy)
33
34 # Mat3d
35 m = phynexis.utils.Mat3d()
36 print("Mat3d:", m)
37 m[0] = phynexis.utils.Vec3d(1.0, 0.0, 0.0)
38 m[1] = phynexis.utils.Vec3d(0.0, 1.0, 0.0)
39 m[2] = phynexis.utils.Vec3d(0.0, 0.0, 1.0)
40 print("identity:", m)

```

Output:

```

1 v: Vec3d(1, 2, 3)
2 len: 3
3 x: 1.0 y: 2.0 z: 3.0
4 v[0]: 1.0 v[1]: 2.0 v[2]: 3.0
5 after mutation: Vec3d(10, 20, 3)
6 a + b: Vec3d(1, 1, 0)
7 a * 2.0: Vec3d(2, 0, 0)
8 2.0 * a: Vec3d(2, 0, 0)
9 dot: 0.0
10 cross: Vec3d(0, 0, 1)
11 Vec3i: Vec3i(1, 2, 3) len: 3
12 Vec6d xx: 1.0 xy: 0.5
13 Mat3d: Mat3d([0, 0, 0], [0, 0, 0], [0, 0, 0])
14 identity: Mat3d([1, 0, 0], [0, 1, 0], [0, 0, 1])

```

Known Issues

- `Vec3u` reports its Python type name as `Vec3i` (the repr also shows `Vec3i`). Functionally it works as unsigned 64-bit integers, but this is a binding quirk.
- `Vec6d` does **not** support `len()`, indexing, or iteration. Only named component access (`xx`, `yy`, etc.) is available.

- `Vec4d` constructor: the C++ order is (w, x, y, z) , but Python passes (x, y, z, w) . The repr output `Vec4d(x, y, z, w)` matches the property names, so `Vec4d(1, 2, 3, 4)` gives $x=2, y=3, z=4, w=1$ which is confusing. Prefer constructing with explicit intent or avoid `Vec4d` unless the w -component semantics are well understood.

Unexposed C++ API

- `VecX<T>` (dynamic-size vector) — used internally, some instantiations not registered for Python
- Matrix arithmetic operators $(+, -, *)$ on `Mat2d`/`Mat3d`
- `data()` pointer access
- `operator+= / -= / *= / /=` compound assignments

Voronoi

C++: `phynexis::utils::voronoi` **Python:** `phynexis.utils.voronoi` **Header:** `src/utils/voronoi/*.hpp`

Voronoi tessellation on surfaces and in volumes. The Python API exposes `Tessellation` (static computation methods) and `Diagram` (result container).

TessellationParams

Configuration for Voronoi computation.

Constructor

`TessellationParams()`

Defaults: `max_iter=1000, tol=0.001, use_cork=True`.

Properties

Property	Type	Description
<code>max_iter</code>	<code>int</code>	Maximum iterations
<code>tol</code>	<code>float</code>	Convergence tolerance
<code>use_cork</code>	<code>bool</code>	Use Cork for boolean clipping

Tessellation

Static methods for computing Voronoi diagrams.

`compute(vt_seeds, stl_model, params)`

Compute Voronoi tessellation from seed points on a surface mesh.

Parameters: | Parameter | Type | Description | |—|—|—| | `vt_seeds` | `list[Vec3d]` | Seed points | | `stl_model` | `STLModel` | Surface mesh | | `params` | `TessellationParams` | Optional parameters |

Returns: `Diagram`

```
compute_centroidal(num_seeds, stl_model, params)
```

Compute centroidal Voronoi tessellation (CVT) with automatic seed placement.

Parameters: | Parameter | Type | Description | |---|---|---| | num_seeds | int | Number of seeds | | stl_model | STLModel | Surface mesh | | params | TessellationParams | Optional parameters |

Returns:Diagram

Diagram

Result container for tessellation (no public constructor).

Example

```
1 import phynexis
2
3 # Load mesh
4 model = phynexis.utils.STLModel()
5 model.init_from_stl("/path/to/sphere.stl")
6
7 # Compute CVT
8 params = phynexis.utils.voronoi.TessellationParams()
9 params.max_iter = 100
10 params.tol = 0.01
11
12 diag = phynexis.utils.voronoi.Tessellation.compute_centroidal(10, model, params)
13 print("diagram:", diag)
```

python

Output:

```
1 diagram: <phynexis.lib.pyutils.voronoi.Diagram object at 0x...>
```

text

Unexposed C++ API

- SphericalDiagram, SphericalTessellation — Partially bound
- Tessellation::compute overload with implicit surfaces

vtk

C++: phynexis::utils::vtk **Python:** phynexis.utils.vtk **Header:** src/utils/vtk/*.hpp

VTK file I/O utilities for reading and writing point data, surface meshes, tetrahedral meshes, and line sets. Supports legacy VTK and VTK-HDF formats.

Data Classes

Class	Description
<code>VtkData</code>	Container for VTK dataset with points, cells, point data, cell data
<code>VtkTopology</code>	Cell connectivity and type information
<code>PointData</code>	Scalar/vector data attached to mesh points
<code>CellData</code>	Scalar/vector data attached to mesh cells

Read Functions

Function	Description
<code>read_points(filename)</code>	Read point cloud from VTK
<code>read_surface(filename)</code>	Read surface mesh from VTK
<code>read_tetrahedra(filename)</code>	Read tetrahedral mesh from VTK
<code>read_lines(filename)</code>	Read line/polygon data from VTK

Write Functions

Function	Description
<code>write_points(filename, points, point_data=None)</code>	Write point cloud to VTK
<code>write_surface(filename, vertices, facets, point_data=None, cell_data=None)</code>	Write surface mesh to VTK
<code>write_tetrahedra(filename, vertices, tets, point_data=None, cell_data=None)</code>	Write tetrahedral mesh to VTK
<code>write_with_lines(filename, vertices, lines, point_data=None)</code>	Write line set to VTK

Format Detection

Function	Description
<code>is_vtkhdf_format(filename)</code>	Check if file is VTK-HDF format
<code>detect_vtkhdf_format(filename)</code>	Detect VTK-HDF format variant

Example

```

1 import phynexis
2
3 # Format detection
4 print("is_vtkhdf:", phynexis.utils.vtk.is_vtkhdf_format("/path/to/data.vtkhdf"))
5
6 # Data classes exist but their constructors and methods may require
7 # additional VecX type registrations for full functionality.
```

python

Known Issues

- `VtkData`, `PointData`, `CellData` constructors and property access may fail if `VecX<T>` return types are not registered.
- Some read/write functions expect `VecX<Vec3d>` or `VecX<int>` arguments which may not be registered.

Unexposed C++ API

- `VtkReader` / `VtkWriter` internal implementations
- `VtkHDF` reader/writer specialized for HDF5 backend
- `format_detection` internals

wrappers

C++: `phynexis::utils::wrappers` **Python:** `phynexis.utils.wrappers` **Header:** `src/utils/wrappers/*.hpp`

Thin wrappers around external libraries (Cork, CGAL, Eigen, igl, TetGen) providing mesh boolean operations, linear algebra, mesh processing, and tetrahedralization.

CorkWrapper

Boolean mesh operations via the Cork library.

Method	Description
<code>mesh_union(m1, m2)</code>	Union of two meshes
<code>mesh_intersect(m1, m2)</code>	Intersection of two meshes
<code>mesh_difference(m1, m2)</code>	Difference ($m1 - m2$)
<code>mesh_xor(m1, m2)</code>	Symmetric difference

Note: `CorkWrapper` has no default constructor exposed. Access methods via the class itself (static/class methods).

EigenWrapper

Linear algebra utilities via Eigen.

Method	Description
<code>solve_linear_eqn(A, b)</code>	Solve $Ax = b$
<code>get_eigen_vector(A, index)</code>	Get eigenvector by index

IGLWrapper

Mesh processing utilities via libigl.

Method	Description
<code>convex_hull(vertices, facets)</code>	Compute convex hull
<code>mesh_decimate(vertices, facets, target)</code>	Mesh simplification
<code>mesh_refine(vertices, facets)</code>	Mesh subdivision
<code>reorient_facets(vertices, facets)</code>	Ensure consistent facet orientation
<code>remove_duplicate_vertices(vertices, facets)</code>	Remove duplicate vertices
<code>remove_unreferenced_vertices(vertices, facets)</code>	Clean unreferenced vertices
<code>marching_cubes(field, corner, spacing, isovalue)</code>	Extract isosurface
<code>mesh_intersect(ray_origin, ray_dir, vertices, facets)</code>	Ray-mesh intersection
<code>facet_components(vertices, facets)</code>	Connected component labeling
<code>check_winding(vertices, facets)</code>	Check winding order
<code>get_points_inside_mesh(points, vertices, facets)</code>	Point-in-mesh test
<code>tetmesh_boundary(vertices, tets)</code>	Extract boundary surface from tetrahedral mesh

CGALWrapper

Computational geometry via CGAL.

Method	Description
<code>get_alpha_shape(points, alpha)</code>	Compute alpha shape
<code>get_tetmesh(vertices, facets)</code>	Tetrahedral mesh generation
<code>smooth_mesh(vertices, facets, iterations)</code>	Mesh smoothing

mpi

MPI wrapper utilities (if MPI is available).

Example

```

1 import phynexis
2
3 # CorkWrapper - access via class methods
4 try:
5     # These require two STLModel instances
6     result = phynexis.utils.wrappers.CorkWrapper.mesh_union(m1, m2)
7 except Exception as e:
8     print("CorkWrapper requires valid mesh inputs")
9
10 # List available methods
11 print("CorkWrapper:", [x for x in dir(phynexis.utils.wrappers.CorkWrapper) if not
12                        x.startswith("_")])
13 print("IGLWrapper:", [x for x in dir(phynexis.utils.wrappers.IGLWrapper) if not
14                       x.startswith("_")])

```

```
13 print("CGALWrapper:", [x for x in dir(phynexis.utils.wrappers.CGALWrapper) if not
    x.startswith("_")])
```

Output:

```
1 CorkWrapper: ['mesh_difference', 'mesh_intersect', 'mesh_union', 'mesh_xor']
2 IGLWrapper: ['check_winding', 'convex_hull', 'facet_components', 'get_points_inside_mesh',
    'marching_cubes', 'mesh_decimate', 'mesh_intersect', 'mesh_refine', 'remove_duplicate_vertices',
    'remove_unreferenced_vertices', 'reorient_facets', 'tetmesh_boundary']
3 CGALWrapper: ['get_alpha_shape', 'get_tetmesh', 'smooth_mesh']
```

Known Issues

- `CorkWrapper` has no default constructor. Methods must be called as `CorkWrapper.mesh_union(...)` rather than creating an instance.
- Most wrapper methods use `VecX<Vec3d>&` (mutable reference) arguments in their lambda bindings. Even though `VecX<T>` `type_caster` is now registered, `pybind11` cannot bind a temporary converted object to a non-const reference. Methods like `IGLWrapper.convex_hull`, `CorkWrapper.mesh_union`, etc. require binding fixes to accept values or use output tuples instead of output reference parameters.
- `TetGenOptions` is exposed but its constructor and properties may have limited binding coverage.

Unexposed C++ API

- `EigenAdapter` / `EigenWrapper` internal matrix conversion helpers
- `mpi` wrapper full MPI collective operations
- `tetgen_get_tetmesh` factory function